Pay What You Spend! Privacy-Aware Real-Time Pricing with High Precision IEEE 754 Floating Point Division

Soumyadyuti Ghosh
Computer Science and Engineering
Indian Institute of Technology Kharagpur
Kharagpur, India
soumyadyuti.ghosh@gmail.com

Boyapally Harishma
Temasek Labs, Nanyang Technological University
Singapore, Singapore
harishma.boyapally@ntu.edu.sg

Ajith Suresh
Technology Innovation Institute
Abu Dhabi, United Arab Emirates
ajith.suresh@tii.ae

Arpita Patra
Indian Institute of Science
Bangalore, India
arpita@iisc.ac.in

Soumyajit Dey
Computer Science and Engineering
Department of Computer Science and Engineering, IIT Kharagpur
Kharagpur, India
soumya@cse.iitkgp.ac.in

Debdeep Mukhopadhyay
Computer Science and Engineering
Department of Computer Science and Engineering, IIT Kharagpur
Kharagpur, India
debdeep.mukhopadhyay@gmail.com

Abstract

Real-Time Pricing (RTP) is a core smart grid functionality that periodically relays pricing signals to end-users based on their fine-grained power consumption data. However, a growing concern revolves around potential information leakage from this consumption data. While existing solutions address this privacy concern by adopting information-theoretic and cryptographic techniques, including differential privacy and homomorphic encryption, we investigate Multi-Party Computation (MPC) to achieve privacy-preserving RTP. Despite extensive research in the MPC domain, the current techniques need to be reinvestigated for the problem at hand. In particular, we highlight that for RTP evaluations, the critical operation is division, where both the numerator and denominator are private. In such cases, the currently reported MPC methods use fixed-point arithmetic, approximations, and precomputations, resulting in large communication overheads and lower computational accuracy. This paper overcomes the state-of-the-art by introducing a novel MPC technique for precise division using IEEE 754 floating-point arithmetic, achieving RTP accuracy up to 10^{-10} for a million consumers, an improvement of at least five orders of magnitude over existing MPC-based approaches. The proposed MPC-based RTP scheme ensures constant communication and computation overhead per consumer and provides formal security guarantees under real-world/ideal-world simulation paradigm. We evaluate the feasibility of our scheme through extensive experiments, including consumer-side validation with an in-house developed smart meter, demonstrating price stabilization and significantly higher accuracy than state-of-the-art MPC solutions.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASIA CCS '25, Hanoi, Vietnam
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1410-8/25/08
https://doi.org/10.1145/3708821.3733913

CCS Concepts

• **Computer systems organization → Embedded and cyber-physical systems**; • **Security and privacy → Privacy-preserving protocols**.

Keywords

Smart Grid, Multi-Party Computation, Real Time Pricing, IEEE-754 Floating Point Representation.

ACM Reference Format:
Soumyadyuti Ghosh, Boyapally Harishma, Ajith Suresh, Arpita Patra, Soumyajit Dey, and Debdeep Mukhopadhyay. 2025. Pay What You Spend! Privacy-Aware Real-Time Pricing with High Precision IEEE 754 Floating Point Division. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '25), August 25–29, 2025, Hanoi, Vietnam.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3708821.3733913

1 Introduction

The smart grid, as a prominent Cyber-Physical System, encompasses power generation and distribution networks, facilitated by a bi-directional communication infrastructure. The real-time fine-grained consumption profiles provided by the consumers are communicated to various entities in the grid to perform diverse operations such as automated demand-response, real-time pricing (RTP), failure detection, etc. Among these, RTP is considered as a core functionality of the smart grid, addressing the limitations of fixed and time-of-use (TOU) tariffs, which can lead to high peak demands, grid overloading, and increased operating costs. RTP can achieve cost savings of up to 34% over traditional TOU tariffs, and up to 47% when combined with Automated Load Management Strategies [57]. Dynamic pricing and demand-response-based control schemes, such as RTP operate in a closed-loop control system, using mutually dependent pricing signals and price-responsive demands to periodically determine a pricing signal while adapting to consumption patterns such that the error between the total market demand and scheduled supply can be reduced [80]. This control-theoretic approach maintains system stability and mitigates the risks of scaling and delay attacks [80], which can potentially lead to

grid inefficiencies, cascading failures, or blackouts [30, 54]. Despite these advancements, RTP schemes work with original fine-grained consumption profiles and pricing signals, resulting in severe privacy threats to the consumers' personal information. Additionally, RTP schemes rely heavily on inverse operations which pose challenges for privacy-preserving RTP computations as errors in secure inverse methods amplify the inaccuracies.

Privacy Concerns. Meter reading streams can unveil an individual appliance's usage pattern, running time, "ON/OFF" status, leading to exposing details of their daily life behavior [50, 79]. For instance, a 2013 case in Naperville showed that 15-minute interval data could disclose waking, leaving, and sleeping times, indicating home occupancy and vulnerability to burglary [79]. In addition to these temporal patterns, various non-intrusive load monitoring algorithms can further compromise user privacy by disaggregating power usage profiles into specific appliance-level consumption [45, 70], further compromising user privacy by revealing detailed personal activities [36]. The National Institute of Standards and Technology also emphasized the privacy risks associated with smart metering data, including unauthorized access and misuse [67].

1.1 Privacy Preserving Pricing

Various secure pricing protocols for smart grids protect consumer privacy through cryptographic approaches, primarily using homomorphic encryption and commitment schemes. Pricing protocols based on Pedersen commitments use elliptic curve cryptography to ensure that consumption data remains private during TOU pricing [32, 51, 75]. In these protocols, consumption values are encrypted through homomorphic commitments, enabling secure aggregation and TOU pricing without revealing individual readings. Similarly, protocols incorporating homomorphic encryption (HE) for privacy-preserving pricing encrypt the consumers' demand profiles and then multiply the encrypted demands by fixed pricing signals to compute the total pricing amounts under traditional TOU tariffs [18, 62, 73, 87]. While these protocols offer consumer privacy, their reliance on static pricing structures limits their utility in dynamic demand-response and real-time pricing systems. Additionally, a usage-based dynamic pricing scheme using HE was proposed [60]. However, it remains vulnerable to insider adversaries, particularly UPs, who, even in a semi-honest capacity, may exploit their access to decrypted data. Moreover, this scheme employs a quadratic pricing function, which does not consider the dynamic, closed-loop requirements essential for RTP, limiting its applicability for interactive grid operations.

Differential Privacy (DP) has also been applied to smart grid privacy [2, 33, 40, 42, 47]. DP schemes usually introduce zero mean Gaussian and Laplacian noise, resulting in minimal pricing errors under TOU pricing. In DP, privacy is governed by the privacy parameter ϵ [39]. A smaller ϵ value indicates stronger privacy but at the cost of greater noise, which can degrade the utility of the output. To alleviate this trade-off, related work [39] proposed optimizing DP by leveraging inherent measurement noise within the RTP system, thereby reducing the additional noise required for achieving ϵ-DP privacy. Nevertheless, as illustrated in Fig. 1, the relative error percentages of pricing signals between original RTP computations and their DP-protected counterparts show that pricing errors can

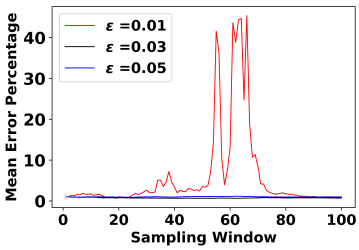


Figure 1: Relative error percentages in pricing signals with $10,000$ **consumers under various privacy levels of differential privacy RTP scheme of [39] compared to IEEE 754** 64**-bit floating point.**

increase up to tenfold for lower ϵ values. Additionally, recent research [38] has demonstrated that consumer home absences can be inferred with high accuracy from DP-protected traces, raising significant privacy concerns. These limitations in existing HE and DP approaches motivate us to explore Multi-Party Computation (MPC) as a potential solution to ensure robust privacy guarantees without sacrificing utility in RTP computations. However, in closed-loop RTP systems, the effect of MPC-based schemes is not well studied in the literature. For example, Abidin et al. [1] proposed a privacy-preserving pricing protocol using secure MPC; but, their approach does not address real-time dynamic pricing or the stability challenges of pricing loops.

1.2 RTP Computation using MPC

In this paper, we adopted an RTP function of the form: $z = f_1(C, \mathcal{S}) \frac{f_2(C,\mathcal{S})}{\sum f_2(C,\mathcal{S})} - f_3(C, \mathcal{S})$ from [80] where $C = (c_1, \ldots, c_n)$ and $\mathcal{S} = (s_1, \ldots, s_m)$. Here, the inputs $(c_1, \ldots, c_n, s_1, \cdots, s_m)$ and the output z are all private values. The RTP computing entities, i.e. miner nodes of the MPC protocol, compute the additive shares of output z by performing basic operations such as addition, multiplication, and division upon receiving the additive shares of the secret inputs $(c_1, \ldots, c_n, s_1, \cdots, s_m)$. Out of these arithmetic operations, division usually requires more computational and communication overheads. Additionally, it is shown in [66, 78] that floating point arithmetic operations cause computational errors. Especially in the presence of division operations, these minor computational errors can cause large deviations in the closed-loop pricing computation leading to unreliable pricing signals [80].

In literature, various secure division protocols for floating point numbers are proposed. The privacy-preserving division protocols based on homomorphic encryption [8, 14, 28, 34, 44, 77, 82] require not only the miner nodes but also the associated input parties to perform complex resource-intensive homomorphic encryption and decryption algorithms. As smart meters are resource-constrained devices, these heavy computations are unsuitable. To address this limitation, various approximation-based inversion and division protocols are proposed that utilize Goldschmidt's method [17, 23, 49, 65] or its variant Markstein's algorithm [63]. They incorporate MPC into their division protocol to guarantee the desired privacy. However, these protocols necessitate multiple iterations to increase the accuracy of the division, leading to increased latency and computational cost.

Upon performing a thorough empirical evaluation of a traditional Goldschmidt's division algorithm without any MPC implementation, we observed that the mechanism can provide accuracy in

the order of 10^{-2} after 10 iterations of computation. MPC-based Goldschmidt's division protocol such as [17] will increase this computational inaccuracy even further due to the presence of more arithmetic operations on the floating point numbers. Moreover, the floating point implementation of the division protocol in [5], which can be found in [43], provides accuracy in the order of 10^{-2} for division operation. The authors of [85] propose a different approximation-based division protocol for machine learning applications that provides significantly higher training accuracy with a relative error of 0.40%; however, the accuracy of the division protocol with floating point numbers is in the order of 10^{-2}. To put into perspective, according to the Energy Information Administration, the average American home uses an average of 29, 130 watts on a daily basis [3]. As a result, an error of 10^{-2} in pricing computation can cause up to 300 units of extra cost per day. It is also evident that as household consumption increases in the coming years, computational errors will have a greater financial impact.

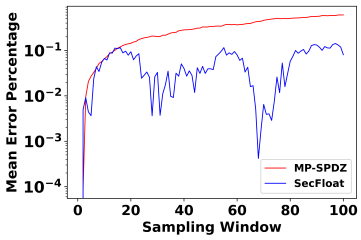


Figure 2: Relative error percentages in pricing signals with 10,000 consumers using MP-SPDZ [55] and SecFloat [72] protocols compared to IEEE 754 64-bit floating point.

The present most accurate state-of-the-art multiparty division protocols, namely MP-SPDZ [55] employing fixed-point arithmetic, along with SecFloat [72] relying on IEEE 754 floating-point arithmetic, exhibit different levels of accuracy when integrated into the RTP computation. As illustrated in Fig. 2, the RTP accuracy for a sample size of 10,000 consumers is in the order of 10^{-1} for MP-SPDZ and 10^{-2} for SecFloat, which can be considered sub-optimal. It is crucial to note that this observed inaccuracy will increase with the number of consumers, as the additional computations required will amplify the errors. The division protocol employed by MP-SPDZ, inspired by Goldschmidt's methodology, results in a trade-off in accuracy. Conversely, SecFloat relies on IEEE 754 floating-point arithmetic for a 32-bit representation, demonstrating a reduction in precision compared to the 64-bit counterpart.

To address these limitations, we propose a low-latency MPC division protocol and integrate a scaling technique to increase the accuracy of the RTP computation up to the order of 10^{-10}, making it more well-suited for error-sensitive applications like RTP.

Contributions. Our novel contributions are as follows:

1. **MPC-based RTP Scheme.** We propose MPC-based RTP protocols for both consumers and suppliers that preserve the stabilization properties of the RTP system and facilitate privacy-preserving computation of consumer prices and supplier revenue based on consumption patterns and scheduled generation (§6.2).

2. **Three-Party Floating Point Division.** We propose a novel and highly accurate floating-point division protocol for a three-party setting under the assumption of an honest majority, where at most one computational party can be semi-honestly corrupted (§5.2). The MPC-based RTP protocol incorporates our division protocol within its framework, ensuring both security and precision.

3. **Security and Performance Evaluation.** We formally prove the security of our proposed protocols using the real-world/ideal-world simulation paradigm (§E). We discuss the efficiency of the proposed division and RTP protocols, which ensure constant communication and computation costs for consumers (§6.3).

4. **Experimental Validation.** We perform extensive experimental validation on our MPC division and RTP protocols, demonstrating higher computational accuracy and efficiency than state-of-the-art protocols such as MP-SPDZ [55], SCALE-MAMBA [4], and SecFloat [72]. Simultaneously, to alleviate computational inaccuracies due to underlying floating-point arithmetic, we propose a scaling mechanism and integrate it with our RTP scheme. This scaling technique increases the accuracy of our MPC-based RTP protocol by more than 10 times (§7.3).

2 Market Models and Real-Time Pricing

In traditional wholesale markets, Utility Providers (UPs) determine supply and prices through a bidding process based on generation and transmission costs [37]. However, UPs can disproportionately influence prices and manipulate market conditions causing distortions and leading to unreliable pricing signals. To mitigate these issues, we adopt the control-theoretic price stabilization model from [80], which utilizes an Independent System Operator (ISO), a neutral entity responsible for balancing supply and demand. The ISO periodically determines and relays pricing signals directly to the users. Real-world examples of ISOs include California Independent System Operator (CAISO), managing 80% of California's electricity [21], PJM, coordinating power for 65 million people across 13 U.S. states [71], and New York Independent System Operator (NYISO), which manages New York's grid [68]. In Europe, ENTSO-E plays a similar role, promoting market integration [35], while Australia's AEMO oversees electricity and gas markets [9].

Below, we present the market models, and the price stabilization scheme of [80]. Such approaches have been widely studied over the years [40, 41, 81, 83]. The market models of [80] had also been employed in previous studies [76] and executed in various experimental RTP programs provided by the UPs [11, 27]. Tab. 1 summarizes essential notations used throughout the paper.

2.1 Supplier Model

In the dynamic pricing scheme, the suppliers schedule their power generation, aiming to maximize their profits. Depending on the power generation, the ISO periodically generates and relays the RTP signals. For any supplier $S_i \in \mathcal{S}$, the scheduled supply $s_{i,k}$ in the kth period can be modeled as $s_{i,k} = p_i \cdot \lambda_k + q_i$, where p_i and q_i are the slope and intercept of the supply model and λ_k is the pricing signal at the kth sampling window. The total scheduled supply in the kth sampling window is denoted as $s_k = \sum_{S_i \in \mathcal{S}} s_{i,k}$.

2.2 Consumer Model

The power consumption of a consumer comprises baseline demand and price-responsive demand. The baseline demand is time-dependent, bounded, and price-independent, while price-responsive demand, $w_{j,k}$, depends on the pricing signal, λ_k. For any consumer

Table 1: Notations used throughout this paper.

Notations	Definition
S_i	The ith supplier for $i \in [1, m]$.
$\lambda^s_{i,k}$	Pricing signal of S_i at the kth period.
λ^s_k	Aggregated pricing signal of suppliers at the kth period.
$s_{i,k}$	Scheduled generation of S_i at the kth period.
s_k	Aggregated generation of suppliers at the kth period.
$e^s_{i,k-1}$	Generation scheduling error of S_i at the kth period.
C_j	The jth consumer for $i \in [1, n]$.
$\lambda^c_{j,k}$	Pricing signal of C_j at the kth period.
λ^c_k	Aggregated pricing signal of consumers at the kth period.
$d_{j,k}$	Power consumption of C_j at the kth period.
d_k	Aggregated consumption of consumers at the kth period.
$e^c_{j,k-1}$	Generation scheduling error of C_j at the kth period.
$(u.\text{s}, u.\text{e}, u.\text{sig})$	(Sign, Exponent, Significand) of u in IEEE-754 representation.
$[\![u]\!]$	IEEE 754 representation of u; $[\![u]\!] = \{u.\text{s}, u.\text{e}, u.\text{sig}\}$.
$[x]$	Secret sharing of x over $\mathbb{Z}_{2^\ell}$.
$[u]^{\text{f}}$	Secret sharing of the floating point value u.
$[\![u[i]]\!]$	Floating point share of u at party P_i.

$C_j \in C$, the price-responsive demand $w_{j,k}$ in the kth period follows the constant elasticity of own-price (CEO) model: $w_{j,k} = D_j \cdot {\lambda_k}^{\epsilon_j}$, where D_j is a positive constant and ϵ_j is the price elasticity of demand. With baseline demand $b_{j,k}$, the total demand for C_j can be modeled a $d_{j,k} = b_{j,k} + w_{j,k}$. Moreover, the total demand of all consumers is represented as d_k.

2.3 Control Theoretic Price Stabilization

The objective of an ISO is to determine a clearing price that equates scheduled supply with overall system demand. This is achieved through a control-theoretic price stabilization algorithm designed to minimize the *generation scheduling error*, $e_k = s_k - d_k$. The ISO uses the previous period's scheduling error to adjust pricing signals, aiming to keep e_k near zero, thereby ensuring market stability. The baseline RTP algorithm presented in [80] is designed to mitigate integrity attacks by ensuring stable pricing signals. This is achieved by the pricing algorithm presented in Eq. 1, where all end users are subjected to the same RTP λ_k.

$$\lambda_k = \lambda_{k-1} - \left(\frac{2\eta}{\dot{s}_0 - \dot{w}_0}\right) \times e_{k-1} \tag{1}$$

Here, $\eta \in (0, 1)$ signifies the price stabilization gain. In Eq. 1, the first derivatives at $k = 0$ for s, w are denoted by $\dot{s}_0$ and $\dot{w}_0$. Eq. 1 ensures that the system can converge to an equilibrium where the generation scheduling error is zero. Hence, for a linearized system, with fixed λ_0, the above RTP algorithm ensures stability. However, this approach does not address privacy concerns for consumers and suppliers. Existing studies [15, 20, 25, 61, 69] show that aggregated demand can be disaggregated to infer household-level consumptions. Applying the same RTP signal to all consumers enables adversaries to infer price-responsive demand by modeling individual responses to the known pricing signal. Using linear approximation methods similar to [80], adversaries can iteratively refine these models to improve estimates of individual demand parameters, potentially compromising consumer privacy. In the next section, we incorporate dynamic pricing functions to improve privacy for both consumers and suppliers.

3 Real Time Customized Pricing Scheme

We adopt the price stabilization algorithm presented in [80] that ensures system stability where the overall generation scheduling error is zero. However, this paper focuses only on the impact of integrity attacks on RTP systems, neglecting consumer usage-based pricing. Dynamic pricing strategies encourage consumers to schedule their consumption to flatten the peak power consumption in the grid [86]. Keeping this in mind, we updated the pricing algorithm presented in [80] to consider dynamic pricing functions for computing consumer/supplier-specific pricing signals in the presence of multiple consumers and suppliers.

Our system consists of n consumers and m suppliers. For the $(k-1)^{th}$ sampling window, the pricing signals for each consumer (C_j) and supplier (S_i) are denoted as $\lambda^c_{j,k-1}$ and $\lambda^s_{i,k-1}$ respectively. Let $\lambda^c_{k-1} = \sum_{j=1}^{n} \lambda^c_{j,k-1}$ and $\lambda^s_{k-1} = \sum_{i=1}^{m} \lambda^s_{i,k-1}$ denote the sum of individual pricing signals for all the consumers and suppliers, respectively. Let d_{k-1} denote the total scheduled demand and s_{k-1} denote the total scheduled supply for the $(k-1)^{th}$ sampling window, where $d_{k-1} = \sum_{j=1}^{n} d_{j,k-1}$ and $s_{k-1} = \sum_{i=1}^{m} s_{i,k-1}$. To compute consumer-specific pricing, we first calculate the generation scheduling error associated with consumer C_j as

$$e^c_{j,k-1} = s_{k-1} \frac{\lambda^c_{j,k-1}}{\lambda^c_{k-1}} - d_{j,k-1} \tag{2}$$

Here, $d_{j,k-1}$ denotes the individual demand of C_j. However, to derive the scheduled generation for C_j, we scale down the overall supply (s_{k-1}) by multiplying it with the ratio of $\lambda^c_{j,k-1}$ and λ^c_{k-1}. We further scale the remaining terms in Eq. 1 to compute the price for the kth sampling window for consumer C_j as presented in Eq. 3. The term $\dot{s}_0$ is divided by the total number of consumers (n) to compute the consumer-specific value. Simultaneously, the term $\dot{w}_0$ is replaced by its respective value $\dot{w}_{j,0}$ for consumer C_j.

$$\lambda^c_{j,k} = \lambda^c_{j,k-1} - \frac{2\eta}{\frac{\dot{s}_0}{n} - \dot{w}_{j,0}} \times e^c_{j,k-1} \tag{3}$$

Similarly, the real-time price for each supplier should vary depending on its power generation. Hence, to derive the supplier-specific price per unit, we first calculate the scheduling error associated with supplier S_i as

$$e^s_{i,k-1} = s_{i,k-1} - d_{k-1} \frac{\lambda^s_{i,k-1}}{\lambda^s_{k-1}} \tag{4}$$

We compute the price per unit for the kth sampling window for supplier S_i in Eq. 5 by scaling the remaining terms in Eq. 1 (similar to Eq. 3).

$$\lambda^s_{i,k} = \lambda^s_{i,k-1} - \frac{2\eta}{\dot{s}_{i,0} - \frac{\dot{w}_0}{m}} \times e^s_{i,k-1} \tag{5}$$

In this way, we derive customized pricing for each consumer and supplier based on their consumption patterns and scheduled generation, respectively. We subsequently implement these dynamic pricing functions (refer to §6.2) within a three-party MPC framework. We discuss the details of our approach next, starting with a brief description of the necessary preliminaries on secure MPC.

4 Secure Multi-Party Computation (MPC)

MPC enables a group of m mutually distrusting parties, $P_1, \cdots, P_{\mathsf{m}}$, each with their respective inputs $x_1, \cdots, x_{\mathsf{m}}$, to jointly compute a public function $f(\cdot)$ on their inputs. MPC ensures that each party P_i learns only its own output $y_i = f(x_1, \cdots, x_{\mathsf{m}})$ and any information that can be inferred from its input and the output, thus ensuring the privacy of the other parties' inputs.

System Model. We consider an MPC setting with $\mathsf{m} = 3$ parties, tolerating up to one semi-honest corruption, operating over an ℓ-bit ring $\mathbb{Z}_{2^\ell}$. The computation parties $\mathcal{P} = P_1, P_2, P_3$ form a 3-party configuration (3PC), known for its scalability in computational tasks and its ability to achieve high throughput in MPC [7, 24], making it well-suited to our real-world application. We follow the 3PC protocol outlined in [16, 17], where each secret $x \in \mathbb{Z}_{2^\ell}$ is *additively* shared among the three parties as $x = x[1] + x[2] + x[3]$, where arithmetic operations are carried over $\mathbb{Z}_{2^\ell}$. We use $[x]$ to denote the additive sharing of x over $\mathbb{Z}_{2^\ell}$ and $x[i]$ denotes the share of party P_i. Additionally, the parties use a one-time key setup based on Pseudo-Random Functions [7, 24], enabling them to sample common random values from $\mathbb{Z}_{2^\ell}$ non-interactively.

Operations. To enable secure and efficient arithmetic over secret shares, we require a set of core MPC primitives in our protocol. Specifically, we adopt the non-interactive resharing protocol $\Pi_{\text{re-sh}}$ and the secure multiplication protocol Π_{mul} from [17], and the Du–Atallah multiplication protocol Π_{DA} from [16]. Tab. 2 provides an overview of the MPC-based operations used in our protocol. Detailed protocol descriptions and their corresponding communication costs are provided in Appendix §A.

Table 2: MPC-based operations over $\mathbb{Z}_{2^\ell}$.

Protocol	Input	Output
Input Sharing	$P_i : x_i$	$[x_i]$
Reshare	$[x]; x$	$[u]; u = x$
Linear	$[x], [y], a, b, c$	$[a \cdot x + b \cdot y + c]$
Multiplication	$[x], [y]$	$[x \cdot y]$
Reconstruction	$[x]$	x

Security. Security of the 3PC protocol in [16, 17] is proven Universally Composable (UC) [22] and, by extension, follows the real-world/ideal-world simulation paradigm. Thus, our constructions build upon these protocols, inherit the same security guarantees.

5 IEEE 754 Floating Point Arithmetic

IEEE 754 [88] is a widely used standard for floating-point arithmetic, providing guidelines for the representation and operations of floating-point numbers. The standard defines the format for finite numbers, infinities, and non-numeric values (NaNs), along with specified arithmetic operations, rounding modes, and error corrections. We particularly focus on the double-precision binary format of IEEE 754, which represents floating-point numbers with 64 bits.

IEEE 754 Standard. IEEE 754 [88] double-precision floating point numbers, represented with the least significant bit b_0 and the most significant bit b_{63}, comprise three components: the sign bit s (b_{63}), the exponent e ($b_{62} \cdots b_{52}$), and the significand sig ($b_{51} \cdots b_0$). The sign bit s determines the number's polarity; if $\mathsf{s} = 1$, the number is negative and is positive otherwise. The eleven bits of exponent represent a number $\mathsf{e} \in [0, \cdots, 2047]$, adjusted by subtracting 1023 to obtain the actual exponent. The significand represents a non-standardized integer $\mathsf{sig} \in [0, \cdots, 2^{52} - 1]$. Special cases arise when exponent is either all zeros or all ones, signifying zero ($\mathsf{e} = 0, \mathsf{sig} = 0$), infinity ($\mathsf{e} = 2047, \mathsf{sig} = 0$), or NaN (Not a Number) ($\mathsf{e} = 2047, \mathsf{sig} \neq 0$). While IEEE 754 serves as the de facto standard for binary floating-point arithmetic, existing implementations may diverge slightly from regular hardware outcomes due to issues like rounding, underflow, and error handling.

5.1 MPC over IEEE 754 Representation

To ensure precise alignment with hardware floating-point procedures, we have adapted the SoftFloat library [48], an IEEE 754 software implementation for floating-point data types. The SoftFloat library provides fundamental functions for floating-point arithmetic: $\mathsf{f}_{\mathsf{add}}$ for addition ($\oplus$), $\mathsf{f}_{\mathsf{sub}}$ for subtraction ($\ominus$), $\mathsf{f}_{\mathsf{mul}}$ for multiplication ($\odot$), and $\mathsf{f}_{\mathsf{div}}$ for division ($\oslash$) [48, 74]. These functions facilitate precise floating-point arithmetic between two numbers, taking input floats (u_1 and u_2) and producing an output (u_3) with similar precision as any hardware implementation.

Modifications to SoftFloat library. To enable MPC computations over floating-point values, we modify the SoftFloat library to embed each of the three components (s, e, sig) in different ring sizes, following existing works [53, 58, 72]. The sign bit (s) is processed within a binary ring ($\mathbb{Z}_2$), ensuring that the polarity of the number is accurately represented and manipulated during operations. The exponent (e) is handled within a 16-bit ring, allowing precise adjustments and calculations that align with the IEEE 754 standard. The significand (sig) is processed within a 64-bit ring, ensuring that the fractional part of the floating-point number is accurately represented and manipulated [72]. This ring-embedded approach extends SoftFloat's IEEE 754 floating-point arithmetic with MPC techniques for secure computations. Leveraging ring-based operations enables efficient, modular arithmetic aligned with hardware implementations. For example, in floating-point addition ($\oplus$), MPC techniques enable secure computation of aligned exponents and significand addition, using ring operations primarily to ensure that computations remain within secure, modular arithmetic constraints.

We use $[\![u]\!] = \{u.\mathsf{s}, u.\mathsf{e}, u.\mathsf{sig}\}$ to denote the IEEE 754 representation of a floating-point value u. In this representation, each operation in the SoftFloat library—$\mathsf{f}_{\mathsf{add}}$, $\mathsf{f}_{\mathsf{sub}}$, $\mathsf{f}_{\mathsf{mul}}$, and $\mathsf{f}_{\mathsf{div}}$—is modified to accept six inputs: $[\![u_1]\!] = \{u_1.\mathsf{s}, u_1.\mathsf{e}, u_1.\mathsf{sig}\}$ and $[\![u_2]\!] = \{u_2.\mathsf{s}, u_2.\mathsf{e}, u_2.\mathsf{sig}\}$. The resulting output is $[\![u_3]\!] = \{u_3.\mathsf{s}, u_3.\mathsf{e}, u_3.\mathsf{sig}\}$, representing the resultant float u_3. While the algorithms for addition, subtraction, multiplication, and division under the IEEE 754 standard remain unchanged [48], the use of ring operations adds layers of security and integrity.

Sharing Semantics. A floating point value u is said to be *secret-shared* in our scheme over the parties in $\mathcal{P}$, if each party $P_i \in \mathcal{P}$ holds $[\![u[i]]\!]$ such that $[\![u]\!] = [\![u[1]]\!] \oplus [\![u[2]]\!] \oplus [\![u[3]]\!]$. We use $[u]^{\mathsf{f}}$ to denote our additive secret-sharing over floating point values. This differs from the secret sharing in SecFloat [72], where $u.\mathsf{s}$, $u.\mathsf{e}$, and $u.\mathsf{sig}$ are individually shared over their respective rings.

Protocol 1 Protocol $\Pi^{\mathsf{f}}_{\mathsf{sh}}(P_c : u)$ to generate $[\cdot]^{\mathsf{f}}$ shares of u.

1: **Input:** Party P_c with input u.
2: **Output:** $[u]^{\mathsf{f}}$ such that $[\![u]\!] = [\![u[1]]\!] \oplus [\![u[2]]\!] \oplus [\![u[3]]\!]$.
3: P_3 chooses uniformly random (s, e, sig) values of $[\![r[1]]\!]$, $[\![r[2]]\!]$, $[\![r[3]]\!]$, respectively.
4: P_3 computes $[\![r]\!]$ s.t. $[\![r]\!] = [\![r[1]]\!] \oplus [\![r[2]]\!] \oplus [\![r[3]]\!]$.
5: P_3 sends (s, e, sig) values of $[\![r[1]]\!]$ to P_1 and $[\![r[2]]\!]$ to P_2.
6: $P_i \in \mathcal{P}$ sends (s, e, sig) values of $[\![r[i]]\!]$ to P_c.
7: P_c computes $[\![(u-r)]\!] = [\![u]\!] \ominus [\![r[1]]\!] \ominus [\![r[2]]\!] \ominus [\![r[3]]\!]$ and sends the masked value $[\![(u-r)]\!]$ to P_1.
8: Parties compute their shares of input as follows:
$P_1 : [\![u[1]]\!] = [\![r[1]]\!] \oplus [\![(u-r)]\!]$
$P_2 : [\![u[2]]\!] = [\![r[2]]\!]$ and $P_3 : [\![u[3]]\!] = [\![r[3]]\!]$.

Input Sharing. The input sharing protocol $\Pi^{\mathsf{f}}_{\mathsf{sh}}$ enables a designated party P_c, holding a floating point secret u, to generate its additive secret shares among parties P_1, P_2, and P_3. For this, protocol $\Pi^{\mathsf{f}}_{\mathsf{sh}}$ uses a standard input sharing method in the outsourced MPC setting, adapted to floating point operations. Specifically, P_1, P_2 and P_3 generate $[\cdot]^{\mathsf{f}}$-shares of a random floating point value $[\![r]\!]$, such that $[\![r]\!] = [\![r[1]]\!] \oplus [\![r[2]]\!] \oplus [\![r[3]]\!]$, which is then opened towards P_c. Party P_c computes $[\![u]\!] \ominus [\![r]\!]$ and sends the result to one of the parties, who adds it to their share of $[\![r]\!]$. The formal protocol $\Pi^{\mathsf{f}}_{\mathsf{sh}}$ is detailed in Protocol 1.

A fundamental challenge in floating-point secret sharing arises from the non-uniform representation of floating-point numbers, where uniform sampling leads to disproportionately large values due to exponent bias in IEEE 754 format [52]. In our 3-party MPC setting, party P_3 uniformly samples the floating-point components (s, e, sig) for $[\![r[1]]\!]$, $[\![r[2]]\!]$, and $[\![r[3]]\!]$ over distinct rings, respectively. Additionally, it is sufficient for party P_c to send $[\![u-r]\!]$ only to P_1. The random sampling of $[\![r]\!]$ by P_3 serves as a uniform mask from P_1's perspective, effectively preventing any leakage of the secret u from the shared value $[\![u-r]\!]$. For practical applications, parties P_1, P_2, and P_3 can exchange their roles across multiple input providers to balance the load and improve efficiency [46, 59].

Other Protocols. To obtain protocols for resharing, addition, and multiplication of floating-point values in our 3PC setting, the parties use the protocols described in §4, but replace standard ring operations with their floating-point counterparts. For example, ring addition is replaced with $\oplus$, and multiplication with $\odot$. The modified protocols are detailed in Appendix §B. An exception is the division protocol over floating point values, for which we have developed an efficient new protocol, detailed next. Additionally, we add the superscript f to the floating point protocols to distinguish them from their ring counterparts. For example, the $\Pi_{\mathsf{re\text{-}sh}}$, Π_{rec}, Π_{add}, Π_{mul}, and Π_{DA} protocols under the IEEE 754 floating-point arithmetic are represented as $\Pi^{\mathsf{f}}_{\mathsf{re\text{-}sh}}$, $\Pi^{\mathsf{f}}_{\mathsf{rec}}$, $\Pi^{\mathsf{f}}_{\mathsf{add}}$, $\Pi^{\mathsf{f}}_{\mathsf{mul}}$, and $\Pi^{\mathsf{f}}_{\mathsf{DA}}$.

5.2 Floating Point Division Protocol

In this section, we present our protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ (Protocol 3) for performing floating-point division in MPC. This protocol is specifically designed for our secret sharing scheme in a three-party setting with one semi-honest corruption. We use the additive secret sharing-based Du-Atallah multiplication protocol ($\Pi^{\mathsf{f}}_{\mathsf{DA}}$) [16] as a key building block for computing the division. The reasons for choosing $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ are detailed in Appendix §C. Additionally, our protocol requires no precomputation and avoids approximation-based techniques.

Protocol 2 Protocol $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}([u]^{\mathsf{f}}, P_k : [\![w]\!])$ to compute $[u \oslash w]^{\mathsf{f}}$.

1: **Input:** $P_i \in \mathcal{P} : [\![u[i]]\!]$ and $P_k : [\![w]\!]$.
2: **Output:** $[u \oslash w]^{\mathsf{f}}$.
3: Party P_k locally computes $[\![y]\!] = [\![1.0]\!] \oslash [\![w]\!]$.
4: Parties locally compute $[z_k]^{\mathsf{f}} = [u[k] \oslash w]^{\mathsf{f}}$ as:
$$P_k : [\![z_k[k]]\!] = [\![u[k]]\!] \odot [\![y]\!] \,,\; P_i \in \mathcal{P} \setminus \{P_k\} : [\![z_k[i]]\!] = [\![0]\!].$$
5: For $P_i \in \mathcal{P} \setminus \{P_k\}$, parties generate $[z_i]^{\mathsf{f}} = [u[i] \oslash w]^{\mathsf{f}}$ as:
$$[z_i]^{\mathsf{f}} = \Pi^{\mathsf{f}}_{\mathsf{DA}}(P_i : [\![u[i]]\!], P_k : [\![y]\!]).$$
6: Parties locally compute $[\cdot]^{\mathsf{f}}$-shares of the output as:
$$[u \oslash w]^{\mathsf{f}} = [z_1]^{\mathsf{f}} \oplus [z_2]^{\mathsf{f}} \oplus [z_3]^{\mathsf{f}}.$$

Given the secret shares of two floating point values, $[u]^{\mathsf{f}}$ and $[v]^{\mathsf{f}}$, the division operation $[u \oslash v]^{\mathsf{f}}$ can be expressed as:

$$[\![u]\!] \oslash [\![v]\!] = ([\![u[1]]\!] \oslash [\![v]\!]) \oplus ([\![u[2]]\!] \oslash [\![v]\!]) \oplus ([\![u[3]]\!] \oslash [\![v]\!]) \tag{6}$$

In our division protocol, we use this observation and proceed with reconstructing a "reciprocal" term of the form $([\![v]\!] \oslash [\![u[j]]\!])$ towards party P_i. This allows P_i to locally compute the reciprocal of the obtained term. For example, in the $\Pi^{\mathsf{f}}_{\mathsf{div}}$ protocol, P_1 obtains $[\![y_3]\!] = [\![v]\!] \oslash [\![u[3]]\!]$ (line 4 in Protocol 3). Consequently, P_1 can locally compute $[\![z[1]]\!] = [\![1.0]\!] \oslash [\![y_3]\!]$, which corresponds to the term $([\![u[3]]\!] \oslash [\![v]\!])$ in Eq. 6. A similar approach is taken for the other two terms in Eq. 6, where values are reconstructed towards P_2 and P_3, respectively. After these steps, the parties possess a $[\cdot]^{\mathsf{f}}$-sharing of the desired output in z (line 5 in Protocol 3). To ensure the final output shares are independent of the inputs, the parties perform a resharing operation ($\Pi^{\mathsf{f}}_{\mathsf{re\text{-}sh}}$) on $[z]^{\mathsf{f}}$ to obtain the final output.

Protocol 3 Protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}([u]^{\mathsf{f}}, [v]^{\mathsf{f}})$ to compute $[u \oslash v]^{\mathsf{f}}$.

1: **Input:** $P_i \in \mathcal{P} : [\![u[i]]\!], [\![v[i]]\!]$.
2: **Output:** $[u \oslash v]^{\mathsf{f}}$.
3: For $i \in \{1, 2, 3\}$, parties execute:
$$[y_i]^{\mathsf{f}} = [v \oslash u[i]]^{\mathsf{f}} = \Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}([v]^{\mathsf{f}}, P_i : [\![u[i]]\!]).$$
4: Parties reconstruct $y_{((i+1)\%3)+1}$ towards $P_i \in \mathcal{P}$ using $\Pi^{\mathsf{f}}_{\mathsf{rec}}$ as:
$$P_i : [\![y_{((i+1)\%3)+1}]\!] \leftarrow \Pi^{\mathsf{f}}_{\mathsf{rec}}([y_{((i+1)\%3)+1}]^{\mathsf{f}}, P_i).$$
5: $P_i \in \mathcal{P}$ locally computes $[\![z[i]]\!]$ for $z = u \oslash v$ as:
$$P_i : [\![z[i]]\!] = [\![1.0]\!] \oslash [\![y_{((i+1)\%3)+1}]\!].$$
6: Parties execute $\Pi^{\mathsf{f}}_{\mathsf{re\text{-}sh}}([z]^{\mathsf{f}})$ to generate $[u \oslash v]^{\mathsf{f}}$.

In the discussion above, the "missing" piece is the reconstruction of the reciprocal term towards each party. For example, in Protocol 3, P_1 obtains $[\![y_3]\!] = [\![v]\!] \oslash [\![u[3]]\!]$. To enable this, we use a helper protocol, $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ (Protocol 2), which computes $[u \oslash w]^{\mathsf{f}}$ given $[u]^{\mathsf{f}}$ and $[\![w]\!]$. Although this looks similar to the division protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$, the main difference here is that the input $[\![w]\!]$ is held in the clear by one of the parties $P_k \in \mathcal{P}$. Note that

$$[\![u]\!] \oslash [\![w]\!] = ([\![u[1]]\!] \oslash [\![w]\!]) \oplus ([\![u[2]]\!] \oslash [\![w]\!]) \oplus ([\![u[3]]\!] \oslash [\![w]\!]) \tag{7}$$

To compute the above equation, party P_k first locally computes the reciprocal of $[\![w]\!]$, denoted by $[\![y]\!]$ (see line 3 in Protocol 2). Then, P_k can compute $([\![u[k]]\!] \oslash [\![w]\!])$ by locally multiplying $[\![u[k]]\!]$ with $[\![y]\!]$. For the remaining two terms, the parties use Du-Atallah

multiplication protocol ($\Pi^{\mathrm{f}}_{\mathrm{DA}}$). Once the parties compute $[\cdot]^{\mathrm{f}}$-shares for all three terms in Eq. 7, they can locally obtain the $[\cdot]^{\mathrm{f}}$-shares of their sum, completing the protocol.

Lemma 5.1. *(Correctness) Secure Floating Point Division: Given the secret shares of two floating point values $[u]^{\mathrm{f}}$ and $[v]^{\mathrm{f}}$, protocol $\Pi^{\mathrm{f}}_{\mathrm{div}}$ correctly computes $[u \oslash v]^{\mathrm{f}}$ in the three-party setting tolerating at most one semi-honest corruption.*

Proof. See Appendix §D for details. □

Security. We prove the security of our division protocol $\Pi^{\mathrm{f}}_{\mathrm{div}}$ (Protocol 3) using the real-world/ideal-world simulation paradigm and the details are provided in Appendix §E.

6 Privacy Preserving Stable RTP Scheme

This section presents the privacy-preserving variant of the Real-Time Customized Pricing Scheme discussed in §3. Here, we detail the system model, corruption strategy, data flow, and our proposed MPC-based RTP protocol.

6.1 Formal Modelling

System and Threat Model. Our proposed model for the RTP system consists of n consumers $\mathcal{C} = \{\mathrm{C}_1, \cdots, \mathrm{C}_n\}$, m suppliers $\mathcal{S} = \{\mathrm{S}_1, \cdots, \mathrm{S}_m\}$, and three ISOs. From the MPC perspective (cf. §4), the three ISOs act as the MPC computation parties P_1, P_2, and P_3. Fig. 3 illustrates a closed-loop RTP system, consisting of individual and building loads on the consumer side, suppliers, and the ISOs. Both consumers and suppliers provide inputs, secret sharing data among the three ISOs.

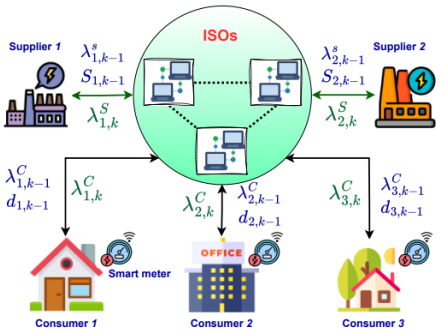


Figure 3: Closed loop Real-Time Pricing System.

We choose three ISOs for MPC-based RTP computation as this approach is widely explored [6, 7, 24, 56, 64] and has been applied in real-world scenarios like private ad measurement systems of Brave [29]. Additionally, the semi-honest corruption model is extensively used in smart metering literature [10, 26, 31, 42, 84], making it a suitable threat model for consumers, suppliers, and ISOs. The primary role of ISOs is to assist consumers and suppliers in computing RTP information while maintaining privacy for all parties. Since ISOs are independent by definition, they do not collude with each other or with suppliers and consumers. All messages among entities are exchanged over authenticated channels. Regarding privacy, suppliers and ISOs may try to learn the individual power consumption ($d_{j,k}$) and price ($\lambda^c_{j,k}$) of the consumer. Similarly, consumers and ISOs may try to learn the individual supply ($s_{i,k}$) and price ($\lambda^s_{i,k}$) of the supplier.

Data Flow. During the kth sampling window, each entity performs the following data operations:

- *Consumer* C_j: The smart meter at consumer $\mathrm{C}_j \in \mathcal{C}$ computes the power consumption data for the $(k-1)^{th}$ sampling window, denoted as $d_{j,k-1}$. C_j generates secret shares of $d_{j,k-1}$ and the pricing signal for the $(k-1)^{th}$ sampling window (denoted as $\lambda^c_{j,k-1}$) among the ISOs.
- *Supplier* S_i: S_i generates secret shares of the produced power (denoted as $s_{i,k-1}$) and the price (denoted as $\lambda^s_{i,k-1}$) for the $(k-1)^{th}$ sampling window among the ISOs.
- *ISO:* Each ISO computes intermediate output shares for each consumer and supplier, then forwards these to the suppliers and consumers for real-time price computation.

As discussed in §3, all inputs and outputs of the RTP pricing function are real numbers. We use IEEE 754 64-bit representation to embed these values and perform secure arithmetic operations on them (see §5 for details).

6.2 MPC-based RTP Scheme

Here, we focus on computing customized prices for both consumers and suppliers. As discussed in §3, the ISOs compute generation scheduling errors $e^c_{j,k-1}$ and $e^s_{i,k-1}$ (cf. Eqs. 2, 4) for $(k-1)^{th}$ period, while preserving input privacy. These errors are securely computed using MPC over IEEE 754 representation, as outlined in §5.

At a high level, the computation begins with consumers and suppliers secret sharing their inputs ($[\cdot]^{\mathrm{f}}$-shares in §5.1) among the three ISOs. The ISOs then perform MPC to calculate the generation scheduling errors for each consumer and supplier in secret-shared form. Once the computation is complete, these secrets are revealed to the respective entities. Upon receiving the scheduling error $e^c_{j,k-1}$, consumer C_j can locally compute the customized price using Eq. 3. Similarly, supplier S_i computes the price according to Eq. 5 upon obtaining the scheduling error $e^s_{i,k-1}$.

Protocol 4 Protocol $\Pi^{\mathrm{f}}_{\mathrm{err\text{-}C}}$ to compute $[e^c_{j,k-1}]^{\mathrm{f}}$ (cf. Eq. 2).

1: **Input:** $[d_{j,k-1}]^{\mathrm{f}}$, $[\lambda^c_{j,k-1}]^{\mathrm{f}}$, $[s_{i,k-1}]^{\mathrm{f}}$; $\forall i \in [1,m], \forall j \in [1,n]$.
2: **Output:** $[e^c_{j,k-1}]^{\mathrm{f}}$ as per Eq. 2.
3: Locally compute $[s_{k-1}]^{\mathrm{f}} = \oplus_{i=1}^{m} [s_{i,k-1}]^{\mathrm{f}}$.
4: Locally compute $[\lambda^c_{k-1}]^{\mathrm{f}} = \oplus_{j=1}^{n} [\lambda^c_{j,k-1}]^{\mathrm{f}}$.
5: Invoke $\Pi^{\mathrm{f}}_{\mathrm{div}}([\lambda^c_{j,k-1}]^{\mathrm{f}}, [\lambda^c_{k-1}]^{\mathrm{f}})$ to generate $[u_j]^{\mathrm{f}}$, where $[\![u_j]\!] = [\![\lambda^c_{j,k-1}]\!] \oslash [\![\lambda^c_{k-1}]\!]$ for $j \in [1,n]$.
6: Invoke $\Pi^{\mathrm{f}}_{\mathrm{mul}}([s_{k-1}]^{\mathrm{f}}, [u_j]^{\mathrm{f}})$ to generate $[v_j]^{\mathrm{f}}$, where $[\![v_j]\!] = [\![s_{k-1}]\!] \odot [\![u_j]\!]$ for $j \in [1,n]$.
7: Locally compute $[e^c_{j,k-1}]^{\mathrm{f}} = [v_j]^{\mathrm{f}} \ominus [d_{j,k-1}]^{\mathrm{f}}$ for $j \in [1,n]$.

Protocol $\Pi^{\mathrm{f}}_{\mathrm{err\text{-}C}}$ in Protocol 4 provides the formal details for the ISOs to securely compute the generation scheduling error $e^c_{j,k-1}$ according to Eq. 2. Similarly, protocol $\Pi^{\mathrm{f}}_{\mathrm{err\text{-}S}}$ in Protocol 5 outlines

the computation of $e^s_{i,k-1}$ as per Eq. 4. For the final price computation at the consumer and supplier, as given in Eq. 3 and Eq. 5, note that the values η, $\dot{s}_{i,0}$, $\dot{s}_0$, $\dot{w}_{j,0}$ and $\dot{w}_0$ (cf. §2 and Tab. 3) are independent of the pricing signals and do not reveal any secret information, thus they can be publicly published. Let

$$T^c_j = \frac{2\eta}{\frac{\dot{s}_0}{n} - \dot{w}_{j,0}} \text{ and } T^s_i = \frac{2\eta}{\dot{s}_{i,0} - \frac{\dot{w}_0}{m}}. \tag{8}$$

Protocol 5 Protocol $\Pi^{\mathsf{f}}_{\text{err-S}}$ to compute $[e^s_{i,k-1}]^{\mathsf{f}}$ (cf. Eq. 4).

1: **Input:** $[d_{j,k-1}]^{\mathsf{f}}$, $[\lambda^s_{i,k-1}]^{\mathsf{f}}$, $[s_{i,k-1}]^{\mathsf{f}}$; $\forall i \in [1,m], \forall j \in [1,n]$.
2: **Output:** $[e^s_{i,k-1}]^{\mathsf{f}}$ as per Eq. 4.
3: Locally compute $[d_{k-1}]^{\mathsf{f}} = \oplus_{j=1}^{n}[d_{j,k-1}]^{\mathsf{f}}$.
4: Locally compute $[\lambda^s_{k-1}]^{\mathsf{f}} = \oplus_{i=1}^{m}[\lambda^s_{i,k-1}]^{\mathsf{f}}$.
5: Invoke $\Pi^{\mathsf{f}}_{\mathsf{div}}([\lambda^s_{i,k-1}]^{\mathsf{f}}, [\lambda^s_{k-1}]^{\mathsf{f}})$ to generate $[u_i]^{\mathsf{f}}$, where $[\![u_i]\!] = [\![\lambda^s_{i,k-1}]\!] \oslash [\![\lambda^s_{k-1}]\!]$ for $i \in [1,m]$.
6: Invoke $\Pi^{\mathsf{f}}_{\mathsf{mul}}([d_{k-1}]^{\mathsf{f}}, [u_i]^{\mathsf{f}})$ to generate $[v_i]^{\mathsf{f}}$, where $[\![v_i]\!] = [\![d_{k-1}]\!] \odot [\![u_i]\!]$ for $i \in [1,m]$.
7: Locally compute $[e^s_{i,k-1}]^{\mathsf{f}} = [s_{i,k-1}]^{\mathsf{f}} \ominus [v_i]^{\mathsf{f}}$ for $i \in [1,m]$.

Then, consumer C_j can precompute T^c_j, and supplier S_i can precompute T^s_i once, ahead of the first sampling window, for better efficiency. Tab. 3 summarizes the public and private values involved in the MPC-based RTP computation with respect to the consumers and suppliers.

Table 3: Public and private parameters of the MPC-based RTP §6.2.

Private Parameters		Public Parameters
Consumer $\mathsf{C}_j \in C$	Supplier $\mathsf{S}_i \in \mathcal{S}$	
$[\![d_{j,k-1}]\!], [\![d_{k-1}]\!],$ $[\![\lambda^c_{j,k-1}]\!], [\![\lambda^c_{k-1}]\!]$	$[\![s_{i,k-1}]\!], [\![s_{k-1}]\!],$ $[\![\lambda^s_{i,k-1}]\!], [\![\lambda^s_{k-1}]\!]$	$\dot{w}_{j,0}, \dot{w}_0,$ $\dot{s}_{i,0}, \dot{s}_0,$ η, n, m

Given the discussion so far, the MPC-based RTP scheme can be summarized as follows:

1: Consumer $\mathsf{C}_j \in C$ generates $[d_{j,k-1}]^{\mathsf{f}}$ and $[\lambda^c_{j,k-1}]^{\mathsf{f}}$ using the input sharing protocol $\Pi^{\mathsf{f}}_{\mathsf{sh}}$ (cf. Protocol 1).
2: Supplier $\mathsf{S}_i \in \mathcal{S}$ generates $[s_{i,k-1}]^{\mathsf{f}}$ and $[\lambda^s_{i,k-1}]^{\mathsf{f}}$ using the input sharing protocol $\Pi^{\mathsf{f}}_{\mathsf{sh}}$ (cf. Protocol 1).
3: ISOs execute $\Pi^{\mathsf{f}}_{\text{err-C}}$ (cf. Protocol 4) to compute $[e^c_{j,k-1}]^{\mathsf{f}}$ (cf. Eq. 2) for each consumer $\mathsf{C}_j \in C$.
4: ISOs execute $\Pi^{\mathsf{f}}_{\text{err-S}}$ (cf. Protocol 5) to compute $[e^s_{i,k-1}]^{\mathsf{f}}$ (cf. Eq. 4) for each supplier $\mathsf{S}_i \in \mathcal{S}$.
5: ISOs reconstruct $[\![e^c_{j,k-1}]\!]$ toward each $\mathsf{C}_j \in C$ using the reconstruction protocol $\Pi^{\mathsf{f}}_{\mathsf{rec}}$ (cf. Appendix §B).
6: ISOs reconstruct $[\![e^s_{i,k-1}]\!]$ toward each $\mathsf{S}_i \in \mathcal{S}$ using the reconstruction protocol $\Pi^{\mathsf{f}}_{\mathsf{rec}}$ (cf. Appendix §B).
7: Consumer $\mathsf{C}_j \in C$ locally computes $[\![\lambda^c_{j,k}]\!]$ using Eq. 3.
8: Supplier $\mathsf{S}_i \in \mathcal{S}$ locally computes $[\![\lambda^s_{i,k}]\!]$ using Eq. 5.

These outlined steps detail the secure computation of real-time prices for consumers and suppliers using MPC. The protocols enable accurate and privacy-preserving pricing calculations without revealing consumer or supplier data, maintaining privacy throughout the computation process.

6.3 Cost Analysis

In each sampling window, the proposed MPC-based RTP scheme maintains constant communication and computation costs for consumers and suppliers, while these costs scale linearly with the number of participants for the ISOs. The following sections detail the communication and computational overheads for these entities.

6.3.1 Computational Complexity. We present the total number of operations performed by each party as follows:

– At the beginning, S_i and C_j compute the constants T^s_i and T^c_j respectively. This is a one-time operation requiring 1 addition and 2 divisions to compute each constant.
– After receiving the $[\cdot]^{\mathsf{f}}$-shares of the inputs from all the suppliers and consumers, ISOs locally compute the $[\cdot]^{\mathsf{f}}$-shares of $[\![s_{k-1}]\!]$, $[\![\lambda^s_{k-1}]\!]$, $[\![d_{k-1}]\!]$, $[\![\lambda^c_{k-1}]\!]$, requiring $(2m+2n-4)$ floating-point addition operations (i.e., $\mathsf{f}_{\mathsf{add}}$). To compute $[s_{k-1} \odot (\lambda^c_{j,k-1} \oslash \lambda^c_{k-1})]^{\mathsf{f}}$, the ISOs execute 1 instance of both $\Pi^{\mathsf{f}}_{\mathsf{mul}}$ and $\Pi^{\mathsf{f}}_{\mathsf{div}}$. Secure multiplication ($\Pi^{\mathsf{f}}_{\mathsf{mul}}$) requires each ISO to perform 8 $\mathsf{f}_{\mathsf{add}}$ and 3 $\mathsf{f}_{\mathsf{mul}}$ operations (including the output re-sharing operation). During the $\Pi^{\mathsf{f}}_{\mathsf{div}}$ protocol, each ISO executes a total of 2 $\mathsf{f}_{\mathsf{div}}$, 9 $\mathsf{f}_{\mathsf{mul}}$, and 26 $\mathsf{f}_{\mathsf{add}}$ operations for each consumer. Additionally, the final share computation of $[\![e^c_{j,k-1}]\!]$ requires 1 $\mathsf{f}_{\mathsf{add}}$ operation. Consequently, for each consumer, each ISO needs to perform 2 $\mathsf{f}_{\mathsf{div}}$, 35 $\mathsf{f}_{\mathsf{add}}$, and 12 $\mathsf{f}_{\mathsf{mul}}$ operations. Similarly, for each supplier, each ISO performs 2 $\mathsf{f}_{\mathsf{div}}$, 35 $\mathsf{f}_{\mathsf{add}}$, and 12 $\mathsf{f}_{\mathsf{mul}}$ operations to derive the final shares of $[\![e^s_{i,k-1}]\!]$. Therefore, for a total of n consumers and m suppliers, each ISO performs $2(m+n)$ $\mathsf{f}_{\mathsf{div}}$, $(37n+37m-4)$ $\mathsf{f}_{\mathsf{add}}$, and $12(n+m)$ $\mathsf{f}_{\mathsf{mul}}$ operations in the kth sampling window.
– Each consumer receives 3 shares from the ISOs for each $e^c_{j,k-1}$ and reconstructs $e^c_{j,k-1}$ using 2 $\mathsf{f}_{\mathsf{add}}$ operations. Similarly, each supplier performs 2 $\mathsf{f}_{\mathsf{add}}$ operations to compute $e^s_{i,k-1}$. Finally, to compute $\lambda^c_{j,k}$ (cf. Eq. 3) and $\lambda^s_{i,k}$ (cf. Eq. 5), C_j and S_i performs 1 $\mathsf{f}_{\mathsf{add}}$ and 1 $\mathsf{f}_{\mathsf{mul}}$ operation. In total, each consumer and supplier performs 3 $\mathsf{f}_{\mathsf{add}}$ and 1 $\mathsf{f}_{\mathsf{mul}}$ operations during the final RTP computation.

6.3.2 Communication Cost. We report the number of messages exchanged per sampling window, divided into three stages:

– *Input Sharing*: Each consumer $\mathsf{C}_j \in C$ generates $[\cdot]^{\mathsf{f}}$-shares for $([\![d_{j,k-1}]\!], [\![\lambda^c_{j,k-1}]\!])$ using two instances of the input sharing protocol $\Pi^{\mathsf{f}}_{\mathsf{sh}}$. In each execution of $\Pi^{\mathsf{f}}_{\mathsf{sh}}$, C_j receives 3 messages from the ISOs and sends 1 message back to one of the ISOs, totaling 4 messages exchanged. Similarly, supplier $\mathsf{S}_i \in \mathcal{S}$ generates $[\cdot]^{\mathsf{f}}$-shares for $([\![s_{i,k-1}]\!], [\![\lambda^s_{i,k-1}]\!])$. Hence, each C_j and S_i exchanges 8 messages during the input sharing phase. Simultaneously, during $\Pi^{\mathsf{f}}_{\mathsf{sh}}$ protocol, the ISO_3 sends 2 messages to ISO_1 and ISO_2 for each secret. As a result, for each consumer and supplier, ISOs engage in a total of 4 message exchanges.
– *Computations by ISOs*: The ISOs perform 1 secure multiplication protocol $\Pi^{\mathsf{f}}_{\mathsf{mul}}$ and 1 $\Pi^{\mathsf{f}}_{\mathsf{div}}$ protocol for each consumer. The $\Pi^{\mathsf{f}}_{\mathsf{mul}}$ protocol requires a total of 6 message exchanges. Similarly, to execute 6 instances of $\Pi^{\mathsf{f}}_{\mathsf{DA}}$, the $\Pi^{\mathsf{f}}_{\mathsf{div}}$ protocol requires 12 message exchanges between the ISOs. Additionally, 6 more message exchanges are required for executing three instances of protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$.

In total, 24 messages are exchanged by the ISOs to perform 1 instance of Π^{f}_{mul} and 1 instance of Π^{f}_{div}. Similarly, they exchange 24 messages during Prot. 5 for each supplier.

- *Output Reconstruction*: Each $C_j \in C$ and each $S_i \in \mathcal{S}$ receives 3 messages from the ISOs to reconstruct $e^{c}_{j,k-1}$ and $e^{s}_{i,k-1}$, respectively, as part of protocol Π^{f}_{rec}.

Therefore, for each sampling window, a total of $(12+24+3)(n+m) = 39(n+m)$ messages are exchanged. For a network with one million consumers, 10 suppliers, 3 ISOs, and a 64-bit floating-point representation, the total communication overhead amounts to approximately 297.6 MB for each sampling window. Over a local area network (LAN) with a speed of 100 Mbps, the transmission time is about 23.8 seconds, while over a wide area network (WAN) with a speed of 10 Mbps, the transmission time increases significantly to around 238.1 seconds, as WAN speeds are typically much lower than LAN speeds, resulting in higher transmission delays.

7 Experimental Results

This section evaluates the proposed MPC-based RTP protocol, focusing on accuracy, computational efficiency, and scalability across large consumer populations. We compare its performance with established frameworks, including SCALE-MAMBA [4], SecFloat [72], and MP-SPDZ [55], highlighting its ability to deliver precise and reliable RTP signals with minimal overhead. Furthermore, we assess the protocol's stability, the impact of consumer load variability, and its feasibility in low-resource smart metering environments.

7.1 Experimental Setup

Our experiments consider up to one million consumers and 10 suppliers. We use the CEO demand model for each consumer C_j, with parameters $D_j \sim \mathcal{N}(7,\ 3.5^2)$ (KW) and $\epsilon_j \sim \mathcal{N}(-0.8,\ 0.1^2)$ (cf. §2.2). Each consumer's baseline load is selected from $[0.276, 0.488]$ KW, based on a scaled version of real data trace as shown in [80]. Initial pricing signals for consumers and suppliers are set to 21 units. We configure our supplier model accordingly by scaling the values of p_i and q_i (cf. §2.1) based on the consumer sizes. We define $p = \sum_{i=1}^{3} p_i$ and $q = \sum_{i=1}^{3} q_i$ such that $p = 30.94$ and $q = 916.68$, as discussed in [80]. The individual values of p_i and q_i are selected randomly while satisfying the overall p and q values, respectively. We use these settings to simulate different experimental phases of this paper under $\eta = 0.5$. The system converges the fastest when $\eta = 0.5$, with the system's pole being at the origin [80].

Experimental Platform: All experiments are conducted on AMD Ryzen 7 5800H (3.20 GHz), 16 GB RAM, GeForce RTX 3060 (4 GB), running Ubuntu 22.04.3 LTS.

7.2 Evaluation Benchmark

We simulate the pricing models (cf. §3) using the specified parameters for $T = 100$ consecutive sampling windows. In each period, we compute the relative error percentage between the privacy-aware pricing signals and the original RTP signals to assess computational accuracy. The original pricing signals for consumer C_j are represented as $\vec{\lambda^{c}_{j}} = \langle \lambda^{c}_{j,1}, \cdots, \lambda^{c}_{j,T} \rangle$, while the privacy-aware signals are $\vec{\lambda^{c'}_{j}} = \langle \lambda^{c'}_{j,1}, \cdots, \lambda^{c'}_{j,T} \rangle$. The error percentage for the ith period, $r^{c}_{j}(i)$, is calculated as: $r^{c}_{j}(i) = \dfrac{|\lambda^{c'}_{j,i} - \lambda^{c}_{j,i}|}{\lambda^{c}_{j,i}} \times 100$. Furthermore, we define $\vec{R^{c}_{j}} = \langle r^{c}_{j}(1), \cdots, r^{c}_{j}(T) \rangle$ as the error vector over T periods, used to evaluate the accuracy of RTP signals. Since the pricing models for consumers and suppliers are similar (cf. §3), this evaluation also applies to suppliers.

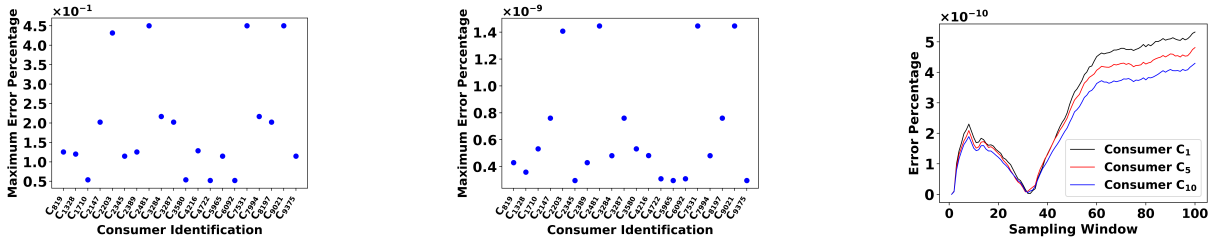
7.3 Evaluation of MPC-based RTP Scheme

This section provides a comparative assessment illustrating significant improvements in accuracy and computational costs of our proposed MPC-based RTP protocol (cf. §6.2) over established protocols such as SCALE-MAMBA [4], SecFloat [72] and MP-SPDZ [55].

Benchmarking SCALE-MAMBA. MP-SPDZ, SCALE-MAMBA, and SecFloat are state-of-the-art frameworks for secure MPC, each excelling in a distinct computational approach. MP-SPDZ employs the fixed-point arithmetic using Goldschmidt's division algorithm, SecFloat relies on IEEE 754 floating-point arithmetic for 32-bit single-precision floating-point operations, and SCALE-MAMBA leverages garbled circuit (GC) for secure computations. These frameworks represent the best solutions within their respective methodologies, offering robust foundations for secure computations.

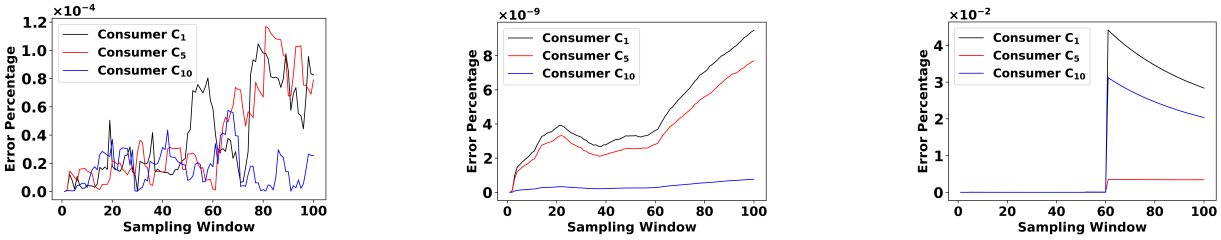
MP-SPDZ and SecFloat exhibit varying levels of precision in RTP computations. MP-SPDZ, which relies on fixed-point arithmetic, shows pricing errors in the order of 10^{-1} under a sample size of one million consumers. SecFloat, which uses IEEE 754-compliant 32-bit floating point arithmetic, achieves the best possible accuracy among existing MPC frameworks, reaching up to in the order of 10^{-3}. Moreover, SCALE-MAMBA, which employs a garbled circuit-based division protocol, performs better than MP-SPDZ but provides lower precision than SecFloat, with errors in the order of 10^{-2}. To enable a fair and controlled comparison, we construct a hybrid benchmark by integrating SCALE-MAMBA's division protocol with our IEEE 754-compliant secure multiplication and addition protocols. We do not construct a hybrid framework based on SecFloat, as it is a two-party protocol, and integrating it with our three-party setting is non-trivial. In contrast, SCALE-MAMBA's garbled circuit-based division integrates smoothly with our framework, allowing us to evaluate a hybrid scheme that combines complex circuit-based primitives with lightweight arithmetic protocols, thereby providing a more diverse and comprehensive validation of our approach. Fig. 5a shows the error percentages in pricing signals for consumers C_1, C_5, and C_{10} using this hybrid configuration. It achieves accuracy in the order of 10^{-4} under a million consumers, outperforming both MP-SPDZ and SecFloat. However, inaccuracies in this approach arise from the complexity of the garbled circuit-based division, which accumulates computational errors and can result in less reliable RTP signals. Due to its relatively strong precision, this hybrid SCALE-MAMBA framework is adopted as the benchmark for evaluating RTP accuracy.

However, garbled circuits are particularly resource-intensive, requiring the generation and evaluation of multiple garbled tables per input bit—leading to substantial communication overhead and latency, which are especially problematic for real-time applications like RTP. Secure transmission and reconstruction of these tables introduce further opportunities for error accumulation. Given these



(a) Using MPC Inverse & Multiplication. (b) Using proposed RTP scheme (§6.2). (c) Using scaling operation in MPC division.

Figure 4: Relative error percentages in pricing signals using our proposed MPC-based RTP scheme (§6.2): (a) maximum error for 20 arbitrary consumer over T sampling windows using MPC inverse and multiplication operation instead of MPC division operation, (b) maximum error for 20 arbitrary consumers over T sampling windows, and (c) error using scaling operation during MPC division protocol.



(a) Using garbled circuits of [4]. (b) Using proposed RTP scheme (§6.2). (c) Using MPC Inverse & Multiplication.

Figure 5: Relative error percentages in pricing signals under a million consumers: (a) using garbled circuit-based division operation of SCALE-MAMBA [4] with our IEEE 754-compliant MPC protocols for multiplication and addition, (b) under the proposed MPC-based RTP scheme (§6.2), and (c) using MPC inverse and multiplication instead of MPC division in the proposed MPC-based RTP scheme (§6.2).

limitations, it is essential to explore alternative methods that offer both high accuracy and computational efficiency. Our proposed MPC-based RTP protocol addresses these challenges by relying on simpler arithmetic operations and lightweight protocols, thereby reducing both computational complexity and cumulative error rates.

Accuracy of Proposed MPC-based RTP Protocol. We evaluate our proposed MPC-based RTP scheme (cf. §6.2) using the obtained $\vec{R}_j^c$ for $\mathsf{C}_j \in \mathcal{C}$. To perform the arithmetic operations on real numbers, we use the IEEE 754 floating-point representation. As evident from Protocol 4 and Protocol 5, the MPC-based RTP scheme requires more arithmetic operations compared to the method that provides no privacy. It is shown in [66, 78] that floating point computations induce computational errors. These induced errors amplify rapidly as the number of arithmetic operations increases. However, Fig. 5b illustrates that the error percentage in pricing signals remains up to the order of 10^{-9} under our MPC-based RTP scheme, considering a million consumers. Our protocol demonstrates superior accuracy, with significantly lower error percentages compared to MPC frameworks like MP-SPDZ, SecFloat (Fig. 2), and hybrid SCALE-MAMBA (Fig. 5a). The use of IEEE 754 floating point representation ensures high precision, and our efficient approach to secure division operations maintains accuracy even in the presence of numerous arithmetic operations. This accuracy is essential for reliable RTP, directly affecting consumer costs and grid stability.

Standard Approaches for Floating Point Division. Unlike the inverse-multiplication method in SCALE-MAMBA, which requires complex preprocessing and multiple communication rounds, in our RTP computation, we use direct division operations (cf. §6.2) simplifying computation and reducing error accumulation. Fig. 5c illustrates the impact of following an inverse-multiplication approach instead of direct division operations. Fig. 4a further supports our findings by showing similar error patterns across all consumers when using the inverse-multiplication methods. The errors are uniformly high, indicating that this approach is fundamentally flawed for our use case. This is because the two-step process amplifies floating point errors, leading to greater inaccuracies. Each step introduces and propagates floating point errors, which are then reflected in the pricing signals. This uniformity in error distribution and lack of precision highlights the importance of our decision to use direct division operations. Fig. 4b demonstrates consistent error rates across different consumers, indicating the robustness and reliability of our protocol. This consistency ensures that no particular consumer experiences disproportionately high errors, which is essential for maintaining fairness and trust in the system. This uniformity also facilitates better planning and management of the grid, as operators can rely on the stability and accuracy of the pricing signals generated by our protocol. The predictable error range further validates the effectiveness of our method in handling large-scale and diverse consumer data without sacrificing accuracy.

Incorporate Scaling Mechanism. Computational errors due to division protocols are particularly common in the MPC domain, as they tend to amplify with the number of operations performed. This phenomenon has already been highlighted in the existing literature [17, 49] and appropriate measures are taken to increase its accuracy. While computing the secure division protocol, the authors of [49] do not explicitly estimate the computation error; rather they iteratively compute the division until the result is guaranteed to be within a specific range. The correct result is then found by a

securely implementing exhaustive search over this range. Though these kinds of approaches provide some reasonable accuracy improvements, they trade off high latency and computational cost. Keeping this in mind, we present a scaling mechanism to increase the accuracy involved in the floating point arithmetic while computing RTP. We observed through our experiments that the Π_{div} protocol induces a maximum amount of errors, due to the presence of floating point inverse operations. Therefore, before performing the Π_{div} protocol, we scale down the shares that are to be inverted during the MPC to $(0, 1]$. Once the secure division is computed, all the values are scaled up to obtain the intended outputs. Choosing the common scaling factor can be considered a problem of finding the maximum of the three scaling factors (corresponding to three ISO's shares). Various MPC-based protocols are proposed in the literature to achieve such functionality. For constant round complexity, multi-party garbled circuit protocol of [12] can be utilized, whereas for efficient implementation, the solutions presented in [13, 19] are more suitable. In our proposed MPC-based RTP, the ISOs scale down the shares of $\lambda^c_{j,k-1}$ and $\lambda^s_{i,k-1}$ to $(0, 1]$ by performing floating point division operation using a common scaling factor before executing Π_{div} protocol. Fig. 4c illustrates the stream $\vec{R}^c_j$ for consumers C_1, C_5 and C_{10} using the above mechanism under 64 bit floating point representation. The error values are reduced by 10 times after the scaling operation compared to Fig. 5b. Here, the obtained pricing signals for the associated consumers are significantly close to that of original pricing signals with error percentage values of order 10^{-10}. This highlights the efficiency of the proposed solution that provides provable privacy guarantees and high computational accuracy.

Consistent RTP Computation. Fig. 6b explores the relationship between the number of consumers in the RTP system and accuracy in pricing signals under our MPC-based RTP scheme. As the number of consumers increases, the error percentage in pricing signals also increases, but at a slower rate compared to other methods. This gradual increase is due to the higher number of floating point operations required for a larger consumer base, which naturally introduces more opportunities for errors. However, our protocol still maintains a lower error rate than existing techniques, showcasing its scalability and efficiency. This ensures that even as the system scales up, the accuracy and reliability of the pricing signals are preserved. This scalability is crucial for modern smart grid applications, where the number of consumers is expected to increase continuously. Our approach contrasts sharply with other methods, such as those using garbled circuits (refer to Fig. 6a), which experience significant increases in both computational complexity and error rates as the consumer base grows.

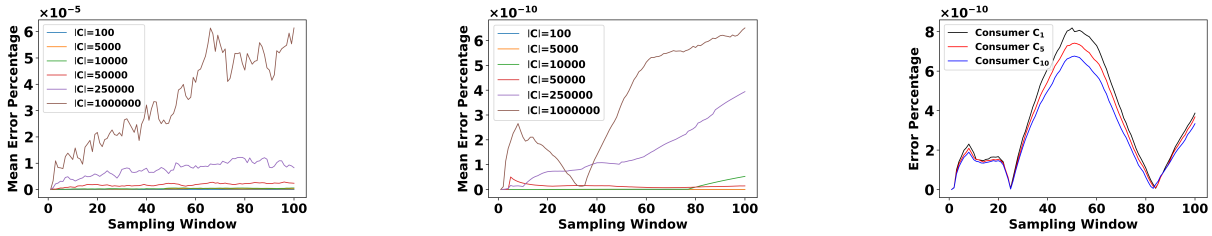
To address real-world consumption complexities, we incorporated time-shifting behaviors for smart appliances, such as EVs and wet appliances, which adjust power usage based on price fluctuations. For example, EVs charge during off-peak hours, and wet appliances operate when electricity is cheaper. We simulated these behaviors by introducing random demand variations in 1000 randomly selected consumers every 10 sampling window. These demand variations simulate real-life scenarios not typically accounted for in the market models described in [76, 80]. As shown in Fig. 6c, our proposed RTP scheme maintains pricing accuracy, even in the presence of these unexpected demand surges.

Table 4: RTP execution time per consumer for different RTP implementation scenarios.

Methodology	Execution time (ms)
SCALE-MAMBA (Garbled Circuit for Division)	6637
MP-SPDZ	4653
SecFloat	190
Proposed RTP Protocol (§6.2)	**30**

RTP System Stability. Our MPC-based RTP protocol provides a reliable and accurate solution, ensuring that the pricing signals remain precise and trustworthy. However, along with the accuracy of pricing signals, it is also important to validate the system stability under our proposed pricing scheme. Fig. 7 compares the error percentages of generation scheduling error (GSE) for our approach against both plaintext RTP computation and the hybrid garbled circuit-based approach. The results reveal significant improvements in GSE accuracy with our protocol, maintaining consistent accuracy across increasing sampling windows. The figure indicates a converging trend where the overall system GSE progressively approaches zero. This convergence is significant as it demonstrates that the system stabilizes effectively using our proposed protocol. Stability in generation scheduling is essential for the power grid's reliability, ensuring that supply meets demand with minimal discrepancies. Our MPC-based protocol manages these discrepancies efficiently, thus maintaining a stable and reliable RTP system. In contrast, the hybrid garbled circuit-based approach shows increasing errors with subsequent sampling windows, highlighting its limitations.

Timing Statistics. Tab. 4 provides a comparative analysis of the RTP execution time per consumer across different MPC protocols, including hybrid SCALE-MAMBA, MP-SPDZ, SecFloat, and our proposed RTP protocol. Our protocol achieves significantly lower execution times ($\approx$ 30.2 ms) in a local network setting, showcasing its computational efficiency through reduced communication overhead and reliance on local computations. In contrast, hybrid SCALE-MAMBA, and MP-SPDZ exhibit higher time complexities due to their reliance on extensive preprocessing, multiple rounds of communication, and complex garbled circuit operations. Moreover, SecFloat requires 190 ms per consumer, which is higher due to increased communication costs and dependency on external operations. By minimizing communication requirements and focusing on local computations, our protocol reduces latency and enhances scalability. This efficiency is particularly advantageous for real-time applications like RTP that require prompt and reliable computations. It is important to note that the execution time is not strictly proportional to the number of consumers, as the majority of the computational time is spent determining the total demand, supply, and pricing signals among the ISO. By reducing both the computational and communication overhead, our approach strikes a balance between computational efficiency and precision. Moreover, our protocol ensures faster and more efficient real-time pricing computations, making it highly suitable for practical deployment in large-scale RTP systems.



(a) Using garbled circuits of [4]. (b) Using proposed RTP scheme (§6.2). (c) Under time-shifting behavior.

Figure 6: Relative error percentage in pricing signals: a) using garbled circuit-based division operation of SCALE-MAMBA [4] with our IEEE 754-compliant MPC protocols, evaluated under increasing consumer sizes (average), b) under our proposed MPC-based RTP scheme with increasing consumer sizes (average), c) using our proposed RTP scheme under time-shifting behaviors of smart appliances.

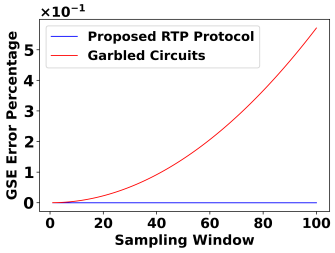


Figure 7: Relative error percentage of generation scheduling errors between proposed MPC-based RTP scheme and using garbled circuit-based division operation of SCALE-MAMBA [4].

Real-time Evaluation with in-house smart meter. Each consumer is equipped with a smart meter that computes and transmits individual power consumption profiles and pricing signals during each sampling window. The purpose of this experiment is to generate time-stamped load consumption profiles and process them in real-time on a low-power single-board computer (SBC), designed for future privacy-enabled metering systems. To achieve this, we built a smart meter using a Raspberry Pi 4 Model B SBC. This setup enables the smart meter to perform secret sharing of demand and pricing data for each consumer, as well as final reconstruction of individual generation scheduling errors and local calculations for precise real-time prices. By running specific protocol components on the smart meter, we assess the practical feasibility of our approach in resource-constrained environments typical of smart metering systems. Further technical details on the smart meter implementation are provided in Appendix §F.

Running the secret-sharing and reconstruction operations alongside RTP calculations on the smart meter confirms that the computational load on input parties remains minimal and manageable in real-world scenarios. Profiling this setup demonstrates an estimated memory overhead of approximately 300 KB, with a runtime per sampling window of approx 1-2 seconds. These results highlight that our protocol can operate efficiently on a resource-constrained device, offering a practical and scalable solution for privacy-preserving real-time pricing systems in modern grids.

7.4 Summary

Our experimental results demonstrate that the proposed MPC-based RTP scheme (§6.2) achieves higher accuracy compared to existing techniques, and maintains RTP system stabilization properties (Fig. 7). The protocol features simpler operations and optimized communication processes that enhance efficiency and scalability without significant computational overheads. The uniform error behaviour across consumers emphasizes its robustness, and the use of additive secret sharing ensures the confidentiality of private data. Compared to garbled circuit-based methods, our protocol demonstrates superior technical performance with lower error rates and faster execution times. These attributes make our protocol highly applicable to real-time smart grid applications, ensuring reliable pricing signals crucial for grid stability and consumer trust.

Artifact Availability. The proof-of-concept implementation codes of our MPC-based RTP protocol and its corresponding results can be found here: MPC-based RTP Protocol.

8 Conclusion

In this paper, we analyzed the impact of MPC-based secure computations on the performance and stability of RTP systems. We developed novel three-party floating point division protocols for the secure and highly accurate computation of these RTP signals. By leveraging IEEE 754 floating point representation and MPC, we effectively addressed the dual challenges of accuracy and privacy. The protocol achieves precision up to the order of 10^{-10}, surpassing methods such as MP-SPDZ [55], SCALE-MAMBA [4], and SecFloat [72], by more than five orders of magnitude. We demonstrated the efficiency of these protocols by ensuring constant communication and computation overheads for the consumers and linear overheads for the ISOs. Our approach ensures more efficient and accurate real-time pricing computations, making it highly suitable for practical deployment in smart grid systems. As potential future work, we plan to enhance the security guarantees of our RTP protocol to provide full protection against malicious adversaries while ensuring data integrity and high accuracy.

Acknowledgments

The authors would like to thank IHUB NTIHAC Foundation, C3i Building at the Indian Institute of Technology Kanpur, the Centre for Hardware-Security Entrepreneurship Research & Development (C-HERD), MeitY, Govt. of India, and Information Security Education and Awareness (ISEA), MeitY, Govt. of India, for partially funding this research. Additionally, Arpita Patra would like to acknowledge the support of the Center of Excellence for Cybersecurity (CySecK) at Indian Institute of Science, Government of Karnataka.

References

[1] Aysajan Abidin, Abdelrahaman Aly, Sara Cleemput, and Mustafa A. Mustafa. 2018. Secure and Privacy-Friendly Local Electricity Trading and Billing in Smart Grid. *ePrint Archive* (2018). http://arxiv.org/abs/1801.08354

[2] Gergely Ács and Claude Castelluccia. 2011. I Have a DREAM! (DiffeRentially privatE smArt Metering). In *Information Hiding (Lecture Notes in Computer Science, Vol. 6958)*. Springer, 118–132.

[3] U.S. Energy Information Administration. 2024. *How much electricity does an American home use?* https://www.eia.gov/tools/faqs/faq.php?id=97&t=3

[4] Abdelrahaman Aly, Karl Cong, Daniele Cozzo, Marcel Keller, Emmanuela Orsini, Dragos Rotaru, Oliver Scherer, Peter Scholl, Nigel P Smart, Titouan Tanguy, and Tim Wood. 2021. SCALE–MAMBA v1.14: Documentation. https://nigelsmart.github.io/SCALE/. Online documentation.

[5] Toshinori Araki, Assi Barak, Jun Furukawa, Marcel Keller, Yehuda Lindell, Kazuma Ohara, and Hikaru Tsuchida. 2018. Generalizing the SPDZ Compiler For Other Protocols. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 880–895.

[6] Toshinori Araki, Assaf Barak, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. 2016. DEMO: High-Throughput Secure Three-Party Computation of Kerberos Ticket Generation. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1841–1843.

[7] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. 2016. High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 805–817.

[8] Mikhail J. Atallah, Marina Bykova, Jiangtao Li, Keith B. Frikken, and Mercan Topkara. 2004. Private collaborative forecasting and benchmarking. In *ACM Workshop on Privacy in the Electronic Society (WPES)*. ACM, 103–114.

[9] Australian Energy Market Operator (AEMO). 2021. 2021 Annual Report. https://aemo.com.au/-/media/files/about_aemo/annual-report/aemo-annual-report-2021.pdf?la=en. Accessed: 2025-05-17.

[10] Haiyong Bao and Rongxing Lu. 2015. A New Differentially Private Data Aggregation With Fault Tolerance for Smart Grid Communications. *IEEE Internet Things J.* 2, 3 (2015), 248–258.

[11] Galen Barbose, Charles Goldman, Ranjit Bharvirkar, Nicole Hopper, Michael Ting, and Bernie Neenan. 2005. *Real Time Pricing as a Default or Optional Service for C&I Customers: A Comparative Analysis of Eight Case Studies.* Technical Report. Ernest Orlando Lawrence Berkeley National Laboratory, Berkeley, CA, USA.

[12] Donald Beaver, Silvio Micali, and Phillip Rogaway. 1990. The Round Complexity of Secure Protocols (Extended Abstract). In *ACM Symposium on Theory of Computing*. ACM, 503–513.

[13] Aner Ben-Efraim, Yehuda Lindell, and Eran Omri. 2016. Optimizing Semi-Honest Secure Multiparty Computation for the Internet. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 578–590.

[14] Tiziano Bianchi, Thijs Veugen, Alessandro Piva, and Mauro Barni. 2009. Processing in the encrypted domain using a composite signal representation: Pros and cons. In *IEEE International Workshop on Information Forensics and Security, (WIFS)*. IEEE, 176–180.

[15] Robert C Blattberg and Subrata K Sen. 1974. Market segmentation using models of multidimensional purchasing behavior: a new segmentation strategy designed to provide better information to the marketing decision maker. *Journal of marketing* 38, 4 (1974), 17–28.

[16] Dan Bogdanov, Sven Laur, and Jan Willemson. 2008. Sharemind: A Framework for Fast Privacy-Preserving Computations. In *European Symposium on Research in Computer Security (Lecture Notes in Computer Science, Vol. 5283)*. Springer, 192–206.

[17] Dan Bogdanov, Margus Niitsoo, Tomas Toft, and Jan Willemson. 2012. High-performance secure multi-party computation for data mining applications. *Int. J. Inf. Sec.* 11, 6 (2012), 403–418.

[18] Fábio Borges, Denise Demirel, Leon Bock, Johannes Buchmann, and Max Mühlhäuser. 2014. A privacy-enhancing protocol that provides in-network data aggregation and verifiable smart meter billing. In *IEEE Symposium on Computers and Communications (ISCC)*. IEEE Computer Society, 1–6.

[19] Lennart Braun, Daniel Demmler, Thomas Schneider, and Oleksandr Tkachenko. 2022. MOTION - A Framework for Mixed-Protocol Multi-Party Computation. *ACM Trans. Priv. Secur.* 25, 2 (2022), 8:1–8:35.

[20] Randolph E Bucklin and Sunil Gupta. 1992. Brand choice, purchase incidence, and segmentation: An integrated modeling approach. *Journal of marketing research* 29, 2 (1992), 201–215.

[21] California Independent System Operator (CAISO). 2021. 2021 Annual Report on Market Issues & Performance. https://www.caiso.com/Documents/2021-Annual-Report-on-Market-Issues-Performance.pdf. Accessed: 2025-05-17.

[22] Ran Canetti. 2001. Universally Composable Security: A New Paradigm for Cryptographic Protocols. In *Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 136–145.

[23] Octavian Catrina and Amitabh Saxena. 2010. Secure Computation with Fixed-Point Numbers. In *Financial Cryptography and Data Security (FC) (Lecture Notes in Computer Science, Vol. 6052)*. Springer, 35–50.

[24] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. 2019. ASTRA: High Throughput 3PC over Rings with Application to Secure Prediction. In *ACM Conference on Cloud Computing Security Workshop (CCSW@CCS)*. ACM, 81–92.

[25] Yuxin Chen and Sha Yang. 2007. Estimating disaggregate models using aggregate data through augmentation of individual choice. *Journal of Marketing Research* 44, 4 (2007), 613–621.

[26] Michael R. Clark and Kenneth M. Hopkinson. 2013. Towards an understanding of the tradeoffs in adversary models of smart grid privacy protocols. In *2013 IEEE Power & Energy Society General Meeting*. IEEE, 1–5.

[27] ComEd 2014. ComEd Residential Real-Time Pricing Program. ComEd. https://hourlypricing.comed.com/.

[28] Morten Dahl, Chao Ning, and Tomas Toft. 2012. On secure two-party integer division. In *International Conference on Financial Cryptography and Data Security*, Vol. 7397. Springer, 164–178.

[29] Alex Davidson, Peter Snyder, E. B. Quirk, Joseph Genereux, Benjamin Livshits, and Hamed Haddadi. 2022. Star: Secret sharing for private threshold aggregation reporting. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 697–710.

[30] GJRE Dileep. 2020. A survey on smart grid technologies and applications. *Renewable energy* 146 (2020), 2589–2625.

[31] Tassos Dimitriou. 2014. Secure and Scalable Aggregation in the Smart Grid. In *International Conference on New Technologies, Mobility and Security (NTMS)*. IEEE, 1–5.

[32] Tom Eccles and Basel Halak. 2017. Performance Analysis of Secure and Private Billing Protocols for Smart Metering. *Cryptography* 1, 3 (2017), 20.

[33] Günther Eibl and Dominik Engel. 2017. Differential privacy for real smart metering data. *Computer Science-Research and Development* 32, 1-2 (2017), 173–182.

[34] Zekeriya Erkin, Martin Franz, Jorge Guajardo, Stefan Katzenbeisser, Inald Lagendijk, and Tomas Toft. 2009. Privacy-Preserving Face Recognition. In *Proceedings on Privacy Enhancing Technologies (PoPETs) (Lecture Notes in Computer Science, Vol. 5672)*. Springer, 235–253.

[35] European Network of Transmission System Operators for Electricity (ENTSO-E). 2021. 2021 Annual Report. https://annualreport2021.entsoe.eu/wp-content/uploads/2022/04/ENTSO-E_Annual_Report_2021.pdf. Accessed: 2025-05-17.

[36] Sören Finster and Ingmar Baumgart. 2015. Privacy-Aware Smart Metering: A Survey. *IEEE communications surveys & tutorials* 17, 2 (2015), 1088–1101.

[37] S-E Fleten and Erling Pettersen. 2005. Constructing bidding curves for a price-taking retailer in the Norwegian electricity market. *IEEE Transactions on Power Systems* 20, 2 (2005), 701–708.

[38] Soumyadyuti Ghosh, Manaar Alam, Soumyajit Dey, and Debdeep Mukhopadhyay. 2024. "Hello? Is There Anybody in There?" Leakage Assessment of Differential Privacy Mechanisms in Smart Metering Infrastructure. In *International Conference on Applied Cryptography and Network Security (ACNS) (Lecture Notes in Computer Science, Vol. 14585)*. Springer, 163–189.

[39] Jairo Giraldo, Alvaro A. Cárdenas, and Murat Kantarcioglu. 2017. Leveraging Unique CPS Properties to Design Better Privacy-Enhancing Algorithms. In *Proceedings of the Hot Topics in Science of Security: Symposium and Bootcamp*. ACM, 1–12.

[40] Jairo Giraldo, Alvaro A. Cárdenas, Murat Kantarcioglu, and Jonathan Katz. 2020. Adversarial Classification Under Differential Privacy. In *Network and Distributed System Security (NDSS)* . The Internet Society.

[41] Jairo Giraldo, Alvaro A. Cárdenas, and Nicanor Quijano. 2017. Integrity Attacks on Real-Time Pricing in Smart Grids: Impact and Countermeasures. *IEEE Transactions on Smart Grid* 8, 5 (2017), 2249–2257.

[42] Matthew Gough, Sérgio F. Santos, Tarek AlSkaif, Mohammad Sadegh Javadi, Rui Castro, and João P. S. Catalão. 2022. Preserving Privacy of Smart Meter Data in a Smart Grid Environment. *IEEE Transactions on Industrial Informatics* 18, 1 (2022), 707–718.

[43] Bar Ilan Cryptography Research Group and NEC Corporation. 2020. *SPDZ-2-With Extensions for Ring Supporting a Floating-point Operation.* https://github.com/nec-mpc/SPDZ-2-Float Accessed: 2025-05-17.

[44] Jorge Guajardo, Bart Mennink, and Berry Schoenmakers. 2010. Modulo Reduction for Paillier Encryptions and Application to Secure Statistical Analysis. In *International Conference on Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 6052)*. Springer, 375–382.

[45] George W. Hart. 1992. Nonintrusive appliance load monitoring. *Proc. IEEE* 80, 12 (1992), 1870–1891.

[46] Christopher Harth-Kitzerow, Ajith Suresh, Yonqing Wang, Hossein Yalame, Georg Carle, and Murali Annavaram. 2025. High-Throughput Secure Multiparty Computation with an Honest Majority in Various Network Settings. *Proceedings on Privacy Enhancing Technologies (PoPETs)* 2025, 1 (2025), 250–272.

[47] Muneeb Ul Hassan, Mubashir Husain Rehmani, Ramamohanarao Kotagiri, Jiekui Zhang, and Jinjun Chen. 2019. Differential privacy for renewable energy resources based smart metering. *J. Parallel and Distrib. Comput.* 131 (2019), 69–80.

[48] J. R. Hauser. 2018. Berkeley SoftFloat (release 3e). http://www.jhauser.us/arithmetic/SoftFloat.html.

[49] Keitaro Hiwatashi, Satsuya Ohata, and Koji Nuida. 2022. An Efficient Secure Division Protocol Using Approximate Multi-Bit Product and New Constant-Round Building Blocks. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* 105-A, 3 (2022), 404–416.

[50] Yuan Hong, Wen Ming Liu, and Lingyu Wang. 2017. Privacy Preserving Smart Meter Streaming Against Information Leakage of Appliance Status. *IEEE Trans. Information Forensics and Security (TIFS)* 12, 9 (2017), 2227–2241.

[51] Marek Jawurek, Martin Johns, and Florian Kerschbaum. 2011. Plug-In Privacy for Smart Metering Billing. In *Proceedings on Privacy Enhancing Technologies (PoPETs) (Lecture Notes in Computer Science, Vol. 6794)*. Springer, 192–210.

[52] David Johnston. 2018. *Random Number Generators—Principles and Practices.* De|G Press. https://doi.org/10.1515/9781501506062 Accessed: 2025-05-17.

[53] Liina Kamm and Jan Willemson. 2015. Secure floating point arithmetic and private satellite collision analysis. *International Journal of Information Security* 14, 6 (2015), 531–548.

[54] Nipendra Kayastha, Dusit Niyato, Ekram Hossain, and Zhu Han. 2014. Smart grid sensor data collection, communication, and networking: a tutorial. *Wireless communications and mobile computing* 14, 11 (2014), 1055–1087.

[55] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1575–1590.

[56] Liisi Kerik, Peeter Laud, and Jaak Randmets. 2016. Optimizing MPC for Robust and Scalable Integer and Floating-Point Arithmetic. In *Financial Cryptography and Data Security: FC 2016 International Workshops, BITCOIN, VOTING, and WAHC, Christ Church, Barbados (Lecture Notes in Computer Science, Vol. 9604)*. Springer, 271–287.

[57] L Klein, J Landeck, L Matos, I Torres, A Bernardes, et al. 2017. Cost-Benefit Comparison of a Time-of-Use Tariff and Real-Time Pricing of Electricity Associated with Automated HVAC Load Management Strategies in Bank Across Mainland Portugal. In *International Conference on Renewable Energies and Power Quality (ICREPQ)*.

[58] Donald Ervin Knuth. 1998. *The art of computer programming, Volume II: Seminumerical Algorithms, 3rd Edition.* Addison-Wesley. https://www.worldcat.org/oclc/312898417

[59] Nishat Koti, Arpita Patra, Rahul Rachuri, and Ajith Suresh. 2022. Tetrad: Actively Secure 4PC for Secure Training and Inference. In *Network and Distributed System Security (NDSS)* . The Internet Society.

[60] Xiaohui Liang, Xu Li, Rongxing Lu, Xiaodong Lin, and Xuemin Shen. 2013. UDP: Usage-Based Dynamic Pricing With Privacy Preservation for Smart Grid. *IEEE Transactions on smart grid* 4, 1 (2013), 141–150.

[61] Jiapeng Liu, Xiuwu Liao, Wei Huang, and Xianzhao Liao. 2019. Market segmentation: A multiple criteria approach combining preference analysis and segmentation decision. *Omega* 83 (2019), 1–13.

[62] Vu Mai and Ibrahim Khalil. 2017. Design and implementation of a secure cloud-based billing model for smart meters as an Internet of things using homomorphic cryptography. *Future Generation Computer Systems* 72 (2017), 327–338.

[63] Peter Markstein. 2004. Software division and square root using Goldschmidt's algorithms. In *Conference on Real Numbers and Computers (RNC'6)*, Vol. 123. Citeseer, 146–157.

[64] Payman Mohassel and Peter Rindal. 2018. ABY^3: A Mixed Protocol Framework for Machine Learning. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 35–52.

[65] Hiraku Morita, Nuttapong Attrapadung, Satsuya Ohata, Koji Nuida, Shota Yamada, Kana Shimizu, Goichiro Hanaoka, and Kiyoshi Asai. 2018. Secure Division Protocol and Applications to Privacy-preserving Chi-squared Tests. In *International Symposium on Information Theory and Its Applications (ISITA)*. IEEE, 530–534.

[66] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. 2018. *Handbook of Floating-Point Arithmetic (2nd Ed.)*. Springer. https://doi.org/10.1007/978-3-319-76526-6

[67] National Institute of Standards and Technology (NIST). 2014. Guidelines for Smart Grid Cybersecurity: Vol. 2, Privacy and the Smart Grid. https://csrc.nist.gov/csrc/media/publications/nistir/7628/rev-1/final/documents/draft_nistir_7628_r1_vol2.pdf. Accessed: 2025-05-17.

[68] New York Independent System Operator (NYISO). 2021. 2021 State of the Market Report. https://www.nyiso.com/documents/20142/2223763/NYISO-2021-SOM-Full-Report-5-11-2022-final.pdf. Accessed: 2025-05-17.

[69] Andy Newing, Graham P Clarke, and Martin Clarke. 2015. Developing and applying a disaggregated retail location model with extended retail demand estimations. *Geographical Analysis* 47, 3 (2015), 219–239.

[70] Dario Piga, Andrea Cominola, Matteo Giuliani, Andrea Castelletti, and Andrea Emilio Rizzoli. 2016. Sparse Optimization for Automated Energy End Use Disaggregation. *IEEE Transactions on Control Systems Technology* 24, 3 (2016), 1044–1051.

[71] PJM Interconnection. 2021. 2021 Annual Report. https://www.pjm.com/m/About-PJM/Who-We-Are/Annual-Report. Accessed: 2025-05-17.

[72] Deevashwer Rathee, Anwesh Bhattacharya, Rahul Sharma, Divya Gupta, Nishanth Chandran, and Aseem Rastogi. 2022. SecFloat: Accurate Floating-Point meets Secure 2-Party Computation. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 576–595.

[73] Alfredo Rial and George Danezis. 2011. Privacy-preserving smart metering. In *ACM workshop on Privacy in the electronic society*. ACM, 49–60.

[74] Anthony Romano. 2014. Practical floating-point tests with integer code. In *International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI) (Lecture Notes in Computer Science, Vol. 8318)*. Springer, 337–356.

[75] Rihem Ben Romdhane, Hamza Hammami, Mohamed Hamdi, and Tai-Hoon Kim. 2021. An Efficient and Privacy-Preserving Billing Protocol for Smart Metering. In *International Conference on Advanced Information Networking and Applications (AINA) (Lecture Notes in Networks and Systems, Vol. 226)*. Springer, 691–702.

[76] Mardavij Roozbehani, Munther A. Dahleh, and Sanjoy K. Mitter. 2012. Volatility of Power Grids under Real-Time Pricing. *IEEE Transactions on Power Systems* 27, 4 (2012), 1926–1940.

[77] Berry Schoenmakers and Pim Tuyls. 2006. Efficient Binary Conversion for Paillier Encrypted Values. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT) (Lecture Notes in Computer Science, Vol. 4004)*. Springer, 522–537.

[78] Alexey Solovyev, Marek S. Baranowski, Ian Briggs, Charles Jacobsen, Zvonimir Rakamaric, and Ganesh Gopalakrishnan. 2019. Rigorous Estimation of Floating-Point Round-Off Errors with Symbolic Taylor Expansions. *ACM Trans. Program. Lang. Syst.* 41, 1 (2019), 2:1–2:39.

[79] SkyVision Solutions. 2013. *Naperville Smart Meters Keep Track of Household Activities.* https://smartgridawareness.org/2013/10/03/smart-meter-data-reveals/

[80] Rui Tan, Varun Badrinath Krishna, David K. Y. Yau, and Zbigniew Kalbarczyk. 2013. Impact of integrity attacks on real-time pricing in smart grids. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 439–450.

[81] David I. Urbina, Jairo Alonso Giraldo, Alvaro A. Cárdenas, Nils Ole Tippenhauer, Junia Valente, Mustafa Amir Faisal, Justin Ruths, Richard Candell, and Henrik Sandberg. 2016. Limiting the Impact of Stealthy Attacks on Industrial Control Systems. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1092–1105.

[82] Thijs Veugen. 2014. Encrypted integer division and secure comparison. *Int. J. Appl. Cryptogr.* 3, 2 (2014), 166–180.

[83] Sreejaya Viswanathan, Rui Tan, and David K. Y. Yau. 2016. Exploiting Power Grid for Accurate and Secure Clock Synchronization in Industrial IoT. In *IEEE Real-Time Systems Symposium (RTSS)*. IEEE Computer Society, 146–156.

[84] Gaurav S. Wagh, Sahil Gupta, and Sumita Mishra. 2020. A distributed privacy preserving framework for the Smart Grid. In *IEEE Power & Energy Society Innovative Smart Grid Technologies Conference (ISGT)*. IEEE, 1–5.

[85] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. 2021. Falcon: Honest-Majority Maliciously Secure Framework for Private Deep Learning. *Proceedings on Privacy Enhancing Technologies (PoPETs)* 2021, 1 (2021), 188–208.

[86] Jiajia Yang, Junhua Zhao, Fushuan Wen, and Zhaoyang Dong. 2019. A Model of Customizing Electricity Retail Prices Based on Load Profile Clustering Analysis. *IEEE Trans. Smart Grid* 10, 3 (2019), 3374–3386.

[87] Yingzhuo Zhao, Qingyu Yang, Donghe Li, and Dou An. 2021. A data aggregation and real-time electricity billing scheme based on homomorphic encryption in smart grid. In *2021 33rd Chinese Control and Decision Conference (CCDC)*. IEEE, 2545–2550.

[88] Dan Zuras, Mike Cowlishaw, Alex Aiken, Matthew Applegate, David Bailey, Steve Bass, Dileep Bhandarkar, Mahesh Bhat, David Bindel, Sylvie Boldo, et al. 2008. IEEE standard for floating-point arithmetic. *IEEE Std 754-2008* (2008). http://doi.org/10.1109/IEEESTD.2008.4610935

A Standard MPC Resharing and Multiplication Protocols

This section provides the details of the MPC protocols discussed in §4. These protocols form the foundational building blocks for achieving secure and efficient arithmetic operations over shared secrets, ensuring correctness while preserving privacy against semi-honest adversaries.

Linear Operations. Given public constants $a, b, c \in \mathbb{Z}_{2^\ell}$ and the secret shares $[x], [y] \in \mathbb{Z}_{2^\ell}$, parties can locally compute the additive shares corresponding to $ax+by+c$ as $a \cdot [x] + b \cdot [y] + c$. For instance, the addition protocol (denoted as Π_{add}) can be seen as a special

case where $a = b = 1$ and $c = 0$. Here, party P_i computes its share of $x + y$ as $x[i] + y[i]$.

Reshare Operation. Given $[x]$, there may be situations where parties need to generate a fresh secret sharing of the same secret x, say u. To achieve this, the parties utilize a non-interactive protocol $\Pi_{\text{re-sh}}$, following the methodology presented in [17].

$[u] \leftarrow \Pi_{\text{re-sh}}([x])$

- $P_i, P_j \in \mathcal{P}$ sample random $r_{ij} \in \mathbb{Z}_{2^\ell}$ where $i < j$.
- P_1 computes $u[1] = x[1] + r_{12} - r_{13}$.
- P_2 computes $u[2] = x[2] + r_{23} - r_{12}$.
- P_3 computes $u[3] = x[3] + r_{13} - r_{23}$.
- **Return** $[u]$.

Multiplication. Given $[x], [y] \in \mathbb{Z}_{2^\ell}$, their product z can be written as $z = xy = \sum_{i=1}^{3} \sum_{j=1}^{3} x[i] \cdot y[j]$. The multiplication protocol in [17] involves distributing input shares among the parties, resulting in a replicated secret sharing where each party holds two out of three shares. This allows the parties to jointly compute all the nine partial products of form $x[i] \cdot y[j]$ necessary for calculating $[z]$. To prevent any information leakage during the process, the parties perform the Reshare operations on the input shares $[x]$ and $[y]$, as well as on the output share $[z']$.

$[z] \leftarrow \Pi_{\text{mul}}([x], [y])$

- $[x'] \leftarrow \Pi_{\text{re-sh}}([x])$, $[y'] \leftarrow \Pi_{\text{re-sh}}([y])$.
- P_1 sends $x'[1]$ and $y'[1]$ to P_2.
- P_2 sends $x'[2]$ and $y'[2]$ to P_3.
- P_3 sends $x'[3]$ and $y'[3]$ to P_1.
- P_1 computes $z'[1] = x'[1]y'[1] + x'[1]y'[3] + x'[3]y'[1]$.
- P_2 computes $z'[2] = x'[2]y'[2] + x'[1]y'[2] + x'[2]y'[1]$.
- P_3 computes $z'[3] = x'[3]y'[3] + x'[2]y'[3] + x'[3]y'[2]$.
- $[z] \leftarrow \Pi_{\text{re-sh}}([z'])$.
- **Return** $[z]$.

Partial Product Computation. Looking ahead (see. §5.2), our protocol requires parties to compute secret shares of partial products as part of RTP computation. Although parties can use a modified version of Π_{mul}, we use a simplified version of the Du-Atallah (Π_{DA}) multiplication protocol [16] for this task, resulting in better communication and accuracy (refer Appendix §C). Given a value $x_i \in \mathbb{Z}_{2^\ell}$ with P_i and $x_j \in \mathbb{Z}_{2^\ell}$ with P_j, the Π_{DA} protocol enables the parties to obtain $[w]$, where $w = x_i \cdot x_j$.

$[w] \leftarrow \Pi_{\text{DA}}(P_i : x_i, P_j : x_j)$

- P_i, P_k samples $\alpha_i \in \mathbb{Z}_{2^\ell}$, while P_j, P_k samples $\alpha_j \in \mathbb{Z}_{2^\ell}$.
- P_i sends $x_i + \alpha_i$ to P_j, while P_j sends $x_j + \alpha_j$ to P_i.
- P_i computes $w'[1] = -(x_i + \alpha_i) \cdot (x_j + \alpha_j) + x_i \cdot (x_j + \alpha_j)$.
- P_j computes $w'[2] = x_j \cdot (x_i + \alpha_i)$.
- P_k computes $w'[3] = \alpha_i \cdot \alpha_j$.
- $[w] \leftarrow \Pi_{\text{re-sh}}([w'])$.
- **Return** $[w]$.

Reconstruction. Protocol Π_{rec} enables parties with shares of $x \in \mathbb{Z}_{2^\ell}$ to reconstruct the secret towards a designated party $P_c \in \mathcal{P}$. To do this, parties send the missing shares of x to P_c. For public reconstruction, this procedure is repeated with each P_i taking on the role of P_c.

Communication Costs. In Π_{mul} protocol, each party P_i sends two shares , $x'[i]$ and $y'[i]$, to the party $P_{(i\%3)+1}$. This results in a total of 6 message exchanges. Since the $\Pi_{\text{re-sh}}$ protocol is non-interactive, the total number of exchanges remains at 6. Conversely, in the Π_{DA} protocol, P_i sends $x_i + \alpha_i$ to P_j, and P_j sends $x_j + \alpha_j$ to P_i, resulting in only 2 message exchanges. Protocol Π_{rec} requires 2 messages for reconstruction towards a party, as each party lacks two out of the three shares of a secret. Similarly, a public reconstruction requires 6 message exchanges.

B Incorporating MPC-based Protocols under the IEEE 754 Floating Point Arithmetic

In this section, we employ the floating-point functions f_{add}, f_{sub}, f_{mul}, and f_{div} (described in §5) to replace the normal operations of MPC-based protocols described in Sec. 4, such as secure addition (Π_{add}) and multiplication protocols (both Π_{mul} and Π_{DA}). Specifically, secure addition (Π_{add}) and secure multiplication protocols Π_{mul} and Π_{DA} as well as the $\Pi_{\text{re-sh}}$ and Π_{rec} protocols are now based on IEEE 754 floating-point arithmetic. In this context, the $\Pi^{\text{f}}_{\text{add}}$ protocol can be represented as: $[\![x]\!] \oplus [\![y]\!] = ([\![x[1]]\!] \oplus [\![y[1]]\!]) \oplus \ldots \oplus ([\![x[n]]\!] \oplus [\![y[n]]\!])$. Here, the output share of the ith computing party in the secure addition protocol between two floating-point numbers $[\![x]\!]$ and $[\![y]\!]$ is computed by performing $\oplus$ operations on their respective shares $[\![x[i]]\!]$ and $[\![y[i]]\!]$: $[\![x[i]]\!] \oplus [\![y[i]]\!]$. Similarly, the $\Pi_{\text{re-sh}}$, Π_{rec}, Π_{mul}, and Π_{DA} protocols are also adjusted accordingly, where traditional addition, subtraction, and multiplication operations are replaced by $\oplus$, $\ominus$, and $\odot$ operations, respectively. The $\Pi_{\text{re-sh}}$, Π_{rec}, Π_{mul}, and Π_{DA} protocols under the IEEE 754 floating-point arithmetic can be formalized as follows. We denote these floating point counterparts as $\Pi^{\text{f}}_{\text{re-sh}}$, $\Pi^{\text{f}}_{\text{rec}}$, $\Pi^{\text{f}}_{\text{mul}}$, and $\Pi^{\text{f}}_{\text{DA}}$ respectively.

$[u]^{\text{f}} \leftarrow \Pi^{\text{f}}_{\text{re-sh}}([x]^{\text{f}})$

- P_i, P_j samples random floating point number $[\![r_{ij}]\!]$, where $i, j \in \{1, 2, 3\}$ and $i < j$.
- P_1 computes $[\![u[1]]\!] = [\![x[1]]\!] \oplus [\![r_{12}]\!] \ominus [\![r_{13}]\!]$.
- P_2 computes $[\![u[2]]\!] = [\![x[2]]\!] \oplus [\![r_{23}]\!] \ominus [\![r_{12}]\!]$.
- P_3 computes $[\![u[3]]\!] = [\![x[3]]\!] \oplus [\![r_{13}]\!] \ominus [\![r_{23}]\!]$.
- **Return** $[u]^{\text{f}}$.

$P_c : [\![x]\!] \leftarrow \Pi^{\text{f}}_{\text{rec}}([x]^{\text{f}}, P_c)$

- Party $P_i \in \mathcal{P} \setminus \{P_c\}$ sends $[\![x[i]]\!]$ to P_c.
- Party P_c computes $[\![x]\!] = [\![x[1]]\!] \oplus [\![x[2]]\!] \oplus [\![x[3]]\!]$.
- **Return** $[\![x]\!]$.

$[z]^{\mathsf{f}} \leftarrow \Pi^{\mathsf{f}}_{\mathsf{mul}}([x]^{\mathsf{f}}, [y]^{\mathsf{f}})$

- $[x']^{\mathsf{f}} \leftarrow \Pi_{\mathsf{re\text{-}sh}}([x]^{\mathsf{f}})$, $[y']^{\mathsf{f}} \leftarrow \Pi_{\mathsf{re\text{-}sh}}([y]^{\mathsf{f}})$.
- P_1 sends $[\![x'[1]]\!]$ and $[\![y'[1]]\!]$ to P_2.
- P_2 sends $[\![x'[2]]\!]$ and $[\![y'[2]]\!]$ to P_3.
- P_3 sends $[\![x'[3]]\!]$ and $[\![y'[3]]\!]$ to P_1.
- P_1 computes $[\![z'[1]]\!] = ([\![x'[1]]\!] \odot [\![y'[1]]\!]) \oplus ([\![x'[1]]\!] \odot [\![y'[3]]\!]) \oplus ([\![x'[3]]\!] \odot [\![y'[1]]\!])$.
- P_2 computes $[\![z'[2]]\!] = ([\![x'[2]]\!] \odot [\![y'[2]]\!]) \oplus ([\![x'[1]]\!] \odot [\![y'[2]]\!]) \oplus ([\![x'[2]]\!] \odot [\![y'[1]]\!])$.
- P_3 computes $[\![z'[3]]\!] = ([\![x'[3]]\!] \odot [\![y'[3]]\!]) \oplus ([\![x'[2]]\!] \odot [\![y'[3]]\!]) \oplus ([\![x'[3]]\!] \odot [\![y'[2]]\!])$.
- $[z]^{\mathsf{f}} \leftarrow \Pi^{\mathsf{f}}_{\mathsf{re\text{-}sh}}([z']^{\mathsf{f}})$.
- **Return** $[z]^{\mathsf{f}}$.

$[w]^{\mathsf{f}} \leftarrow \Pi^{\mathsf{f}}_{\mathsf{DA}}(P_i : [\![x_i]\!], P_j : [\![x_j]\!])$

- P_i, P_k samples random $[\![\alpha_i]\!]$, while P_j, P_k samples random $[\![\alpha_j]\!]$.
- P_i sends $[\![x_i]\!] \oplus [\![\alpha_i]\!]$ to P_j, while P_j sends $[\![x_j]\!] \oplus [\![\alpha_j]\!]$ to P_i.
- P_i computes $[\![w'[1]]\!] = \ominus([\![x_i]\!] \oplus [\![\alpha_i]\!]) \odot ([\![x_j]\!] \oplus [\![\alpha_j]\!]) \oplus [\![x_i]\!] \odot ([\![x_j]\!] \oplus [\![\alpha_j]\!])$.
- P_j computes $[\![w'[2]]\!] = [\![x_j]\!] \odot ([\![x_i]\!] \oplus [\![\alpha_i]\!])$.
- P_k computes $[\![w'[3]]\!] = [\![\alpha_i]\!] \odot [\![\alpha_j]\!]$.
- $[w]^{\mathsf{f}} \leftarrow \Pi^{\mathsf{f}}_{\mathsf{re\text{-}sh}}([w']^{\mathsf{f}})$.
- **Return** $[w]^{\mathsf{f}}$.

C Justification for Selecting the $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ Multiplication Protocol in Secure Division Protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$

In this appendix, we provide a detailed explanation for our choice of the additive secret sharing-based secure Du-Atallah ($\Pi^{\mathsf{f}}_{\mathsf{DA}}$) multiplication protocol, as presented in [16], as the foundational component for performing secure division protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$. The $\Pi^{\mathsf{f}}_{\mathsf{div}}$ protocol comprises 3 instances of the $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ protocol, with each $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ instance performing 2 $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocols per computational party. Hence, the $\Pi^{\mathsf{f}}_{\mathsf{div}}$ protocol involves a total of 6 $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocols. Executing 6 instances of the $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocol requires a total of 12 message exchanges (refer to §4). If the $\Pi^{\mathsf{f}}_{\mathsf{mul}}$ protocol were employed instead of the $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocol, a total of 36 message exchanges would be necessary to perform 6 instances of protocol $\Pi^{\mathsf{f}}_{\mathsf{mul}}$. This comparison highlights that incorporating the $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocol into $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ significantly enhances the performance of the division protocol. Furthermore, the computational accuracy of the $\Pi^{\mathsf{f}}_{\mathsf{div}}$ protocol is also comparatively higher (order of 10^{-16} against 10^{-14}) when the $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocol is used during its construction. For example, when $x = 114.57687$ and $y = 335187.76502$ are considered to compute $\frac{x}{y}$ using $\Pi^{\mathsf{f}}_{\mathsf{mul}}$ protocol, the relative error of the output with respect to 64-bit floating point arithmetic is $1.63125e-14$. However, when $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocol is considered for secure division, the relative error is comparatively lower i.e. $1.5793e-16$. Based on these observations, we have developed the division protocol incorporating the $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocol to ensure both efficiency and higher computational accuracy.

D Proof of Correctness of $\Pi^{\mathsf{f}}_{\mathsf{div}}$ Protocol

This section presents the proof of correctness for the secure floating point division protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$, as stated in Lemma 5.1.

Proof. The proof of correctness for the division protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ can be derived from the following observation. Here, the term $[\![u]\!] \oslash [\![v]\!]$ can be decomposed as: $[\![u]\!] \oslash [\![v]\!] = ([\![u[1]]\!] \oslash [\![v]\!]) \oplus ([\![u[2]]\!] \oslash [\![v]\!]) \oplus ([\![u[3]]\!] \oslash [\![v]\!])$. Without loss of generality, to illustrate, we consider $T_1 = [\![u[1]]\!] \oslash [\![v]\!]$ and express T_1 as:

$$[\![u[1]]\!] \oslash [\![v]\!] = [\![1]\!] \oslash \Big(([\![v[1]]\!] \oslash [\![u[1]]\!]) \oplus ([\![v[2]]\!] \oslash [\![u[1]]\!]) \oplus ([\![v[3]]\!] \oslash [\![u[1]]\!]) \Big)$$

Here, the terms $([\![v[1]]\!] \oslash [\![u[1]]\!])$, $([\![v[2]]\!] \oslash [\![u[1]]\!])$ and $([\![v[3]]\!] \oslash [\![u[1]]\!])$ when added using floating point addition operation ($\oplus$) computes $[\![v]\!] \oslash [\![u[1]]\!]$. In this context, the first term $([\![v[1]]\!] \oslash [\![u[1]]\!])$ is computed locally by P_1 as shown in lines 3-4 of protocol $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$. The remaining terms $([\![v[2]]\!] \oslash [\![u[1]]\!])$ and $([\![v[3]]\!] \oslash [\![u[1]]\!])$ are obtained by executing 2 instances of $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ protocol as shown in line 5 of $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$. The parties $P_i \in \mathcal{P}$ then perform floating point addition operation ($\oplus$) on their respective shares to obtain the shares of $([\![v]\!] \oslash [\![u[1]]\!])$. Upon obtaining these shares protocol $\Pi^{\mathsf{f}}_{\mathsf{rec}}$ is executed toward the party P_2 to obtain $([\![v]\!] \oslash [\![u[1]]\!])$, as shown in line 4 of $\Pi^{\mathsf{f}}_{\mathsf{div}}$. P_2 then performs a floating point division operation ($\oslash$), to obtain $([\![u[1]]\!] \oslash [\![v]\!])$. Similarly, the parties execute 2 more instances of $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ (line 3 of $\Pi^{\mathsf{f}}_{\mathsf{div}}$), followed by protocol $\Pi^{\mathsf{f}}_{\mathsf{rec}}$ and floating-point operation $\oslash$ to obtain $([\![u[2]]\!] \oslash [\![v]\!])$ and $([\![u[3]]\!] \oslash [\![v]\!])$ respectively. The final value of $[\![u]\!] \oslash [\![v]\!]$ can be obtained by performing floating point addition on these shares. This proves the correctness of the division protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$, as it ensures that the computed shares, when combined, accurately represent the division of the secret floating point numbers $[\![u]\!]$ and $[\![v]\!]$. Thus, the protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ correctly and securely computes the division operation within the MPC framework. □

E Security Analysis

We prove the security of our MPC-based RTP scheme using the real-world/ideal-world simulation paradigm. As the building blocks $\Pi^{\mathsf{f}}_{\mathsf{add}}$, $\Pi^{\mathsf{f}}_{\mathsf{DA}}$, and $\Pi^{\mathsf{f}}_{\mathsf{mul}}$ used in our protocol are already proven to be secure in prior works (refer to Sec. 4), we focus on the security of our division protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ (Protocol 3) presented in Sec. 5.2.

Ideal Functionality $\mathcal{F}_{\mathsf{div}}$ for computing $[u/v]^{\mathsf{f}}$.

$\mathcal{F}_{\mathsf{div}}$ receives the secret-shares of IEEE 754 compatible floating point inputs u and v from honest parties and from adversary $\mathcal{A}$ on behalf of the corrupt party. Upon receiving the shares,

- $\mathcal{F}_{\mathsf{div}}$ reconstructs the secrets u and v and computes $w = u/v$ using IEEE 754 floating-point division.
- $\mathcal{F}_{\mathsf{div}}$ computes $[w]^{\mathsf{f}}$ and distributes it among the parties P_1, P_2, and P_3 as per the sharing semantics.

Lemma E.1. *(Security) Secure Floating Point Division: Given the secret shares of two floating point values $[u]^{\mathsf{f}}$ and $[v]^{\mathsf{f}}$, protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ (Protocol 3) securely computes $[u \oslash v]^{\mathsf{f}}$ in the three-party setting*

tolerating at most one semi-honest corruption and achieves security under the real-world/ideal-world simulation paradigm.

Proof. Let $\mathcal{A}$ be the semi-honest adversary corrupting one of the parties. We aim to demonstrate that for every adversary $\mathcal{A}$ in the real world, there exists a simulator $\mathcal{S}$ in the ideal world that produces a view for the adversary that is computationally indistinguishable from the view in the real protocol execution. Without loss of generality, we assume $\mathcal{A}$ corrupts P_1 and provide details for the corresponding simulator, denoted by $\mathcal{S}_{P_1}$. The cases for $\mathcal{A}$ corrupting P_2 and P_3 are similar as their roles are symmetric and hence we omit those details.

As protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ (Protocol 3) internally invokes $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ (Protocol 2), we provide the simulation for $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ first. Specifically, for each step in $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$, simulator $\mathcal{S}_{P_1}$ proceeds as follows:

- **Lines 3-4:** $\mathcal{S}_{P_1}$ simulates local inversion and multiplication operations by computing $[\![y]\!]$ and $[z_k]^{\mathsf{f}}$, using actual shares from P_1 and random values that fit the expected distributions for P_2 and P_3.
- **Line 5:** For each call to $\Pi^{\mathsf{f}}_{\mathsf{DA}}$, $\mathcal{S}_{P_1}$ invokes the simulator for $\Pi^{\mathsf{f}}_{\mathsf{DA}}$, ensuring that all simulated multiplicative interactions are indistinguishable from those what P_1 would observe in the real protocol. Similarly, additions are simulated using the $\Pi^{\mathsf{f}}_{\mathsf{add}}$ protocol's simulator, ensuring the additive shares are consistent with the protocol's expectations and the adversary's view.
- **Line 6:** The shares are combined to simulate the output shares $[u \oslash w]^{\mathsf{f}}$. $\mathcal{S}_{P_1}$ ensure that the randomness in the simulated messages aligns perfectly with the randomness of the actual protocol execution to maintain indistinguishability.

Given the simulation of $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$, we now proceed with simulating Protocol Π_{div}:

- **Line 3:** $\mathcal{S}_{P_1}$ executes three instances of Protocol $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ to simulate shares resulting from division operations for each $[\![u[i]]\!]$ and $[\![v[i]]\!]$, ensuring consistency with real-world execution.
- **Lines 4-5:** $\mathcal{S}_{P_1}$ invokes the simulator for $\Pi^{\mathsf{f}}_{\mathsf{rec}}$ where communication of shares of $[y_i]^{\mathsf{f}}$ among the parties are simulated. In this step, $\mathcal{S}_{P_1}$ has to reconstruct y_3 to $\mathcal{A}$ on behalf of corrupt P_1. For this, $\mathcal{S}_{P_1}$ uses the information it obtained during simulation of $\Pi^{\mathsf{f}}_{\mathsf{p\text{-}div}}$ to ensure that the redistribution of shares matches the protocol's expectations and secures against the inference of the actual values. For each P_i, the simulator combines shares to compute local inversion, simulating the secure computation of $[\![z[i]]\!] = [\![u[i] \oslash v]\!]$ using random dummy values.

The simulation completes with $\mathcal{S}_{P_1}$ invoking simulator for $\Pi^{\mathsf{f}}_{\mathsf{re\text{-}sh}}$, where the shares from the previous step are re-shared, matching the distribution of the actual protocol execution.

By assumption, the sub-protocols $\Pi^{\mathsf{f}}_{\mathsf{DA}}$ and $\Pi^{\mathsf{f}}_{\mathsf{add}}$ are secure and hence provide simulation-based security. Given that $\mathcal{S}_{P_1}$ simulates all interactions and internal computations that $\mathcal{A}$ can observe, the view of $\mathcal{A}$ in the ideal world is computationally indistinguishable from the view in the real world. Concretely, no PPT adversary $\mathcal{A}$ can distinguish between the two worlds with non-negligible probability. Therefore, the protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ is secure in the real-world/ideal-world simulation-based security paradigm. This ensures that even if one of the three parties is semi-honestly corrupt, protocol $\Pi^{\mathsf{f}}_{\mathsf{div}}$ is secure. □

F Smart Meter Setup Details

The in-house smart meter test bed consists of a Raspberry Pi 4 Model B SBC, powered by a quad-core Cortex-A72 (ARM v8) 64-bit processor. The system includes a ZMCT103C current transformer for real-time load current measurement, paired with an 8-channel, 10-bit Analog-to-Digital Converter (ADC), the MCP3008, to convert analog signals into digital format. The smart meter employs a zero-crossing detection mechanism to capture peak values from each cycle of the current waveform, subsequently calculating power consumption by averaging these values every 0.5 seconds. The data is displayed on an LCD connected to the Raspberry Pi via SPI using the wiringPi library. As shown in Fig. 8, we provide one power consumption measurement of 60W load using our in-house metering device to illustrate the reliability of the device.



Figure 8: Power measurement of 60W using our in-house meter.